

1. What is a Data Structure?

Ans. A data structure is a way of organizing and storing data in a computer's memory that one can effectively process data to be searched, traversed, or modified. By using data structure complex problems can be solved effectively. One can use linear data structure(Array, Linked List, queue, stack), or non-linear data structure(Tree, Graph) for Logical or mathematical computation.

For Example Student entities have many attributes just like student_id, student_name, roll_no that will be stored in the database as information of student and fetched, modified, and searched along with specific details.

2. What are the resources of algorithm analysis?

Ans. An algorithm can be analysed using two resources.

1. Time
2. Space

Both are the criteria used to analyse the algorithm.

The time is measured by the no. of processes and space is measured by the CPU memory needed by the algorithm.

3. What is the classification of data structure?

Ans. **The data structure can be classified in two ways.**

1. Primitive data structure
2. Non-primitive data structure

Non-primitive data structure again can be classified as a linear data structure and non –linear data structure.

4. What do you mean by linear data structure?

A sequence of elements is called a linear data structure. For example Arrays, Linked List, Queue, Stacks.

5. What do you mean by non-linear data structure?

When multiple elements are organized into a hierarchical relationship then that structure of data is called a non-linear data structure. For example Tree, Graph

6. Write an algorithm to swap two variables?

tep1: Read a=10, b=20, t

Step2: t=a,a=b,b=t

Step 3: print a, b;

Step4: stop

7. What is the difference between the algorithm and flowchart?

An algorithm is a step-by-step representation of a given problem or a program. A flowchart is a graphical representation of the sequence of operations or a program.

8. What do you mean by pseudocode?

Pseudocode is an informal way such as everyday English that helps the programmer to develop algorithms.

9. What do you mean by the complexity of the algorithms?

he analysis and complexity of an algorithm by which the algorithm is best for a problem are determined by the complexity of an algorithm. Problem is determined as searching and algorithm are defined by the linear search or a binary search.

10. What is the complexity of an algorithm?

To analyse an algorithm we can use two criteria.

1. Time Complexity:

Time complexity is the amount of time used by the processor(CPU) to run the process for execution.

2. Space Complexity:

The space complexity of an algorithm is the specific amount of memory for program execution.

For example:

$$M=f(n)$$

n: is the no. of input

M: running time and space requirement of an algorithm to process data.

11. On what basis time complexity would be analysed?

Number of array elements accessed, number of comparisons through a critical loop, and no. of operations to be performed.

12. What is an array?

Ans. A list of similar data elements at a finite number(n) is called an array. An array is a collection of a variable of the same type the elements of the array are referenced by an index.

Length of an array=UB-LB+1

UB: Upper Bound(Largest index)

LB: Lower Bound (Smallest Index)

Array is of two types:

1. Single Dimensional array:

Syntax: `int Array[10];`

2. Multi Dimensional array:

Syntax: `int array[10][10];`

(two dimensional array)

`int array [10][10][10];`

(multi dimensional array)

LB

UB

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

13. What is the computational complexity of linear search?

The computational complexity of the linear search is the maximum number of comparisons you need to search the array. Linear search is also called sequential search.

For Example: if an array of 100 elements then the maximum number of comparison required is $n-1$, 99 comparison may be required.

14. What is Binary search?

Ans. The binary search algorithm can be used with a sorted array only. After each comparison being searched in the array eliminates one-half of the elements. If the search key is less than the middle element of the array, the first half of the array is searched, If it's not, in the specified subarray the algorithm is repeated one-quarter of the original array.

SKEY < A[MID] then

Set END = MID - 1

Set START = MID + 1

MID = (START + END) / 2

If SKEY = A[MID]

Set LOC = MID

Else

LOC = -1

return LOC

EXI

15. How to read array input from user?

Ans.

```
1  int    A[10][10];

2  int i,j;

3  for(i=0;i<10;i++)

4  {

5  for(j=1;j<=10;j++)

6  {

7  printf("enter array elements");

8  scanf("%d" , &A[i][j]);

9  }

10

11 }
```

16. What do you mean by sorting?

Ans.

Sorting is the process of arranging data items in an ascending or descending order. There are different types of algorithms to sort an element or an array-like: merge sort, bubble sort, selection sort, etc.

17. What is a Linked list?

Ans.

A linked list is a linear data structure. It is a linear collection of data elements called a node pointing to the next node. Node is divided into two parts the first part contains the information of the elements and the second part contains the address of the next node in the list.

The Head is a special pointer variable that contains the address of the first node of the list if there is no node available in the list then, the Head contains a NULL value that means it's empty.

The next pointer of the last node is the NULL pointer signal at the end of the list.

Linked lists are dynamic so the length of the list can be increased or decrease if necessary.

Learn Here, How to Ace [Linked List Interviews](#).

18. What is a doubly-linked list?

Ans. In the doubly Linked list, all nodes are linked together by multiple links which help in accessing both the next node and the previous node. A doubly Linked list helps to traverse the list in the towards the direction and backward direction.

19. What is the difference between a singly linked list and an array?

Ans.

Array	Singly-linked list
Fixed size(static)	Not have a fixed size(dynamic)
Elements are stored in linear order accessible with an index.	Elements are stored in linear order, accessible with links.
Types of the array:1.Single-dimensional array2.multi-dimensional array	Types of linked-list1.singly-linked list2.Doubly linked list3.Circular linked list4.circular-doubly linked list

20. What do you mean by a circular doubly linked list?

Ans.

A circular doubly linked list has both the successor pointer(next node) and predecessor pointer(previous node) in a circular manner.

21. Write a traversing algorithm for the linked list?

Ans.

Step1: PTR=HEAD

Step2: repeat Step 3 and step 4 until PTR!=NULL

Step3: PTR->INFO

Step4: PTR=PTR->NEXT

END of Step2

Step 5: stop

22. What do you mean by memory allocation?

Ans.

Memory allocation is of two types:

1. Static memory allocation: it is also called compile-time allocation using arrays
2. Dynamic memory allocation: it is also called run time allocation using pointers.

23. What do you mean by new and delete operator?

Ans: **New operator:**

Dynamic memory is allocated using the new operator.

Syntax:

```
ptr=new int;
```

```
ptr=new int [10];
```

Delete operator:

Once the memory is no longer needed it can be release for other requests of dynamic memory this is done by the delete operator.

Syntax:

delete ptr;

delete [] ptr;

24. What is Stack in Data Structure ?

Ans. A stack is a list of elements in which an element could be added or deleted only at one end, called the top of the stack. Stack operations are performed on the concept of LIFO (Last in First out).

Two operations associated with the stack:

1. Push(): is the term used to insert an element into the stack.
2. Pop(): is the term used to delete an element from the stack.

Two ways to represent stack into the memory is:

1. Array (static)
2. Linked List (Dynamic)

If top = -1 or NULL represents that stack is empty

25. Write an algorithm for push operation of stack data structure?

Ans.

PUSH(STACK, TOP, STACKSIZE, ITEM)

Step1: if TOP = STACKSIZE - 1 then

Step2: TOP = TOP + 1

Step3: $STACK[TOP]=ITEM$

Step4: return

26. Write an algorithm for pop operation?

Ans.

$POP(STACK, TOP, ITEM)$

Step1: if $top=-1$ then

Print: UNDERFLOW/stack is empty and return

Step2: set $ITEM=STACK[TOP]$

Step3: $TOP=TOP-1$

Step4: return

27. What is a Queue in Data Structure ?

Ans.

The queue is a linear data structure. queue permits insertion of an element at one end and deletion of an element at another end. The end at which insertion can take place is called the rear and the end at which the deletion of an element takes place is called the front.

A queue is worked on the principle FIFO(First in First out).

Enqueue(insertion) and Dequeue(deletion) are two primary operations performed on the queue data structure.

Array(static) and Linked List(dynamic) are two ways to represent a queue in the memory.

“Front” and “Rear” are two pointer variables. Front=NULL indicates the queue is empty.

NOTE: whenever an element is deleted from the queue, the value of Front increased by 1.

Front=Front+1

Whenever an element is added to the queue, the value of Rear is increased by 1

Rear=Rear+1

28. Write an algorithm for insertion in a queue?

Ans.

Step1: if REAR=(MAXSIZE-1) then

Print: queue is overflow and return

Step2: READ NUM (Inserted into the queue)

Step3: REAR=REAR+1

Step4: set QUEUE[REAR]=NUM

Step5: If FRONT=-1 then

Set FRONT=0

Step6: stop

29. Write an algorithm for deletion in a queue?

Ans.

Step1: if FRONT=-1 then

Print: "queue is underflow" and return

Step2: set num:=QUEUE[FRONT]

STEP3: print: "deleted item is: num"

Step4: set FRONT=FRONT+1

Step5: if Front>REAR then

Set FRONT:=REAR=-1

Step6:stop

30. What do you mean by Dequeue?

Ans. A double-ended queue is a linear list in which elements can be added or removed at either one end but not in the middle.

31. Write an algorithm for bubble sort?

Ans.

Let a[5] is an array

Step1: read 5 elements of an array

Step2: print an array A

Step3: Repeat step3 to 4 i=0 to 3

Step4:repeat step4 for j=0to 4-i

If $A[j] > A[j+1]$

Set $TEMP = A[j+1]$

Set $A[j+1] = A[j]$

Set $A[j] = TEMP$

END of if

END of step4

END of step 3

Step5:print sorted array A

Step6: exit

32. Write down the complexity of various sorting algorithms?

Ans.

S.N.	Algorithm	worst-case	average-case	best-case

1	Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
2	Radix sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
3	Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
4	Bubble sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
5	Quicksort	$O(n^2)$	$\log_2 n$	$\log_2 n$
6	Insertion sort	$O(n^2)$	$O(n^2)$	$n-1$
7	Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

33. What do you mean by hashing?

Ans. Hashing is a searching technique also called hash addressing that is used to compute the location of the desired record to retrieve the desired record in single access.

34. What do you mean by the hash table?

Ans. The Hash table is divided into several buckets and each bucket is then capable of storing several records in a data structure where we store a key-value after applying the hash function.

35. What do you mean by hash function?

Ans. A hash function is defined as a function that takes “key “ as input and transforms it into the hash index.

H: $K \rightarrow M$

H is a hash function

K is a set of keys

M is a set of memory addresses.

36. What do you mean by trees?

Ans. tree is a non-linear data structure. A tree is used to represent the hierarchical (parent-child) relationship existing between data items. Items are arranged in a sorted sequence. For the example File system and DOM Structure.

37. What do you mean by a binary tree?

Ans. binary tree is a special type of tree in which every node can have at least a maximum of 2 children.

A binary tree data structure can be represented using two methods:

1. Array representation
2. Linked List representation
3. The maximum number of nodes in a binary tree of height h is $2^h - 1$.

There are three types of binary tree traversals:

1. In order(left-root-right): The left node visited first, then the root node is visited and later we go for visiting the right child node.
2. Pre order(root-left-right): The root node is visited first then the left child and later its right child.
3. Post order (left-right-root): The left child is visited first, then its right child, and then its root node.

38. What do you mean by binary search tree?

Ans. BST is a type of binary tree in which for each node, the value of all the nodes in the left subtree is lesser or equal and the value of all the nodes in the right subtree is greater. Time complexity to find an element in BST is $O(\log_2 n)$.

39. Explain traversing of a binary tree with example expression is $:(p+q)^*(4-3)$

Ans. Inorder: $p+q*4-3$

Preorder: $*+pq-43$

Postorder: $pq+43-*$

40. What do you mean by AVL Tree?

Ans. An AVL tree is a self-balancing binary search tree if the difference between the height of left and right child of every node in the tree is either 0, -1, +1 a binary tree is called balanced.

41. What do you mean by the graph?

Ans. A graph is a non-linear data structure. A graph is a collection of nodes(also called vertices) connected by edges(E) each edge connects two nodes.

$G=(V, E)$ G is a graph that is an ordered pair of V and E.

Graphs are of two types:

1. Directed graph(unidirectional):Edges have direction.
2. Undirected graph(Bidirectional): Edges have no-direction.

42. What do you mean by a complete graph?

Ans. A complete graph that has a maximum number of edges. For a directed graph with n vertices the maximum number of edges $=n(n-1)$. For an Undirected graph with n vertices the maximum number of edges $=n(n-1)/2$.

43. What do you mean by graph traversal techniques?

Ans. Graph traversal is used for searching a vertex in a graph and also used to measure the order of vertices to visit in search.

There are two graph traversal algorithms:

1. Breadth-first search: BFS is a graph search algorithm that begins with the root node and explores all the child nodes until it finds the specified nodes.

The time complexity of BFS is $O(|E|+|V|)$

2. Depth-first search: DFS expanding the sorting node and then going deeper until the element or node is not found.

The time complexity of DFS is $O(|E|+|V|)$

44. Define recursion?

Ans. Recursion is a function that is partially defined by itself. recursion makes code smaller.

45. Write down the vertices & edges in this graph?

Ans. $V=\{A,B,C,D,E\}$

$E=\{AB,BC,CE,ED,AD,BD\}$

46. What do you mean by linear search

Ans. Linear search is mostly used to search an unordered list of elements (where data elements are not sorted).

47. What do you mean by AVL rotation?

Ans. To make itself balanced an AVL tree may perform four kinds of rotations.

1. Left rotation
2. Right rotation
3. Left-right rotation
4. Right-left rotation

48. Which data structure is used to perform recursion?

Ans. STACK because of the concept of LIFO (Last in First out)

49. What do you mean by heap?

Ans. A balanced binary tree has a special case called heap where the root-node key is compared with its children and arranged accordingly.

If p has child node r then

$Key(p) \geq key(r)$

Heap can be of two types:

1. Min-heap: where the value of the root node is less than or equal to either of its child node.
2. Max-heap: where the value of the root node is greater than or equal to either of its child node.

50. What do you mean by insertion sorting?

Ans. Insertion sorting is implemented by inserting a particular element at the appropriate position.

51. What do you mean by quick sort?

Ans. The basic strategy of quicksort is divide and conquer. It is faster and easier to sort two small arrays than one large array. Quicksort can sort a List of data elements faster than any of the common sorting algorithms.

52. What do you mean by external sorting?

Ans. External sorting is applied to the huge amount of data that can not be in the memory all at a time.

53. What do you mean by internal sorting?

Ans. When all the data that is to be sorted can be accommodated at a time in memory then internal sort methods such as bubble sort, merge sort, heap sort, etc are used.

54. What is the primitive data structure?

Ans. Integer, Boolean, Character, Float are the primitive data structure.

55. What is the linear data structure?

Ans. Arrays, Linked List, Queues, Stacks

56. What is the non-linear data structure?

Ans. Tree, Graph.

57. What do you mean by traversing?

Ans. Traversing means accessing each node exactly once so that certain items in the record may be processed.

58. What are the data structure operations?

Ans.

1. Traversing
2. Searching
3. Inserting
4. Deleting

59. What do you mean by merging?

Ans. Combining the two sorted records of two different files into a single file is called merging.

Welcome to our comprehensive guide for acing your Object-Oriented Programming (OOP) interviews in 2026!

In this article, we've curated the top 70 OOPs interview questions and answers to help you master the most critical concepts and techniques.

Whether you're brushing up on your OOP viva questions or gearing up for a career-defining interview, these selected object oriented programming interview questions cover everything from basic principles to advanced topics.

Top 10 OOPs Interview Questions in 2026

Great Learning has compiled the top 10 Object-Oriented Programming (OOP) interview questions commonly encountered during interviews.

- What is OOPs?
- What is the difference between Procedural programming and OOPs?
- Why use OOPs?
- What are the basic concepts of OOPs?
- What is Encapsulation?
- What is Abstraction?
- What is method overloading?
- What is method overriding?
- Types of Inheritance in OOPS
- What is an object?

This blog is further divided into 3 different sections, they are :

- Basic OOPs concepts interview questions
- OOPs Interview Questions for Experienced
- Frequently Asked OOP interview questions

Let's dive in

Basic OOPs Interview Questions

1. What is OOPs?

Object-Oriented Programming (OOP) is a programming approach that employs classes and objects to organize software programs into reusable code templates.

Languages such as Java, C++, Python, and JavaScript support OOP by facilitating the creation of classes that specify attributes and methods, while objects are instances of these classes.

OOP principles encompass inheritance, encapsulation, polymorphism, and abstraction, facilitating the representation of real-world entities in software design.

Don't just learn Java; master it with our free "[OOPS in Java Free Course](#)."

2. Difference between Procedural programming and OOPs?

Procedural Programming	Oops
Procedural Programming is based on functions.	Object-oriented programming is based on real-world objects.
It shows the data to the entire program.	It encapsulates the data.
It does not have a scope for code reuse.	It provides more scope for code reuse.
It follows the concept of top-down programming.	It follows a bottom-up programming paradigm.
The nature of the language is complicated.	It is less complicated in nature, so it is easier to modify, extend and maintain.

It is hard to modify, extend and maintain the code.	
---	--

3. Why use OOPs?

Modularity: OOP promotes modular design, making dividing complex problems into smaller, manageable parts easier.

Reusability: With OOP, you can use classes and objects repeatedly in different parts of a program or other programs, saving time and making programs more efficient.

Encapsulation: Encapsulation hides the inner workings of an object and only shows what's necessary, which keeps things secure and prevents accidental changes.

Inheritance: OOP lets you create new classes based on existing ones, which helps reuse code and build organized relationships between classes.

Polymorphism: OOP allows different objects to be treated as belonging to the same group, making software more flexible and more accessible to expand.

Abstraction: OOP simplifies real-world things into easier-to-understand models, focusing on what's essential and hiding unnecessary details.

4. What are the basic features of OOPs?

The basic features of OOPs are:

- Inheritance
- Encapsulation
- Polymorphism

- Abstraction

5. What is Encapsulation?

Encapsulation in object-oriented programming involves bundling data and methods within a class, restricting direct access to the data from outside the class. This concept offers several benefits:

Data Protection: Encapsulation shields data from unauthorized access, preventing unintended modification and ensuring data integrity.

Modularity: It promotes modular design by encapsulating related data and methods within a single unit, enhancing code organization and maintainability.

Simplified Interface: Encapsulation provides a straightforward interface for interacting with objects, allowing users to focus on the functionality rather than the implementation details.

Flexibility: By hiding internal implementation details, encapsulation enables developers to modify the internal structure of a class without affecting its external interface, promoting code reuse and reducing dependencies.

6. What is Abstraction?

Abstraction in object-oriented programming involves simplifying complex systems by focusing on essential characteristics while hiding unnecessary details. It allows developers to create models that represent real-world entities more understandably.

```
1  class Car:

2      def __init__(self, make, model, year):

3          self.make = make

4          self.model = model

5          self.year = year

6

7      def start_engine(self):

8          # Code to start the car's engine

9          pass

10

11     def drive(self):

12         # Code to drive the car

13         pass

14

15 # Creating an instance of the Car class
```

```
16 my_car = Car("Toyota", "Camry", 2022)
```

In this example, the 'Car' class represents the abstraction of a car.

It encapsulates a car's essential attributes and behaviors, such as its make, model, and year, as well as methods like 'start_engine()' and 'drive()'.

This abstraction allows us to work with cars in our code without needing to understand all the intricate details of how they work internally.

7. What is method overloading?

[Method overloading](#) is a feature in object-oriented programming that allows a class to have multiple methods with the same name but different parameters or parameter types.

This enables developers to define various versions of a technique, each tailored to accept other types or numbers of parameters.

When a method is called, the appropriate version is selected based on the parameters provided.

For example, consider a Calculator class with an add() method:

```
1 public class Calculator {  
  
2     public int add(int num1, int num2) {  
  
3         return num1 + num2;  
  
4     }
```

5

6 **public double** add(**double** num1, **double** num2) {

7 **return** num1 + num2;

8 }

9 }

In this example, the add() method is overloaded to accept integer and double parameters.

Depending on whether integer or double values are passed as arguments, the corresponding version of the add() method will be invoked.

This allows for flexibility and convenience when working with different data types.

8. What is method overriding?

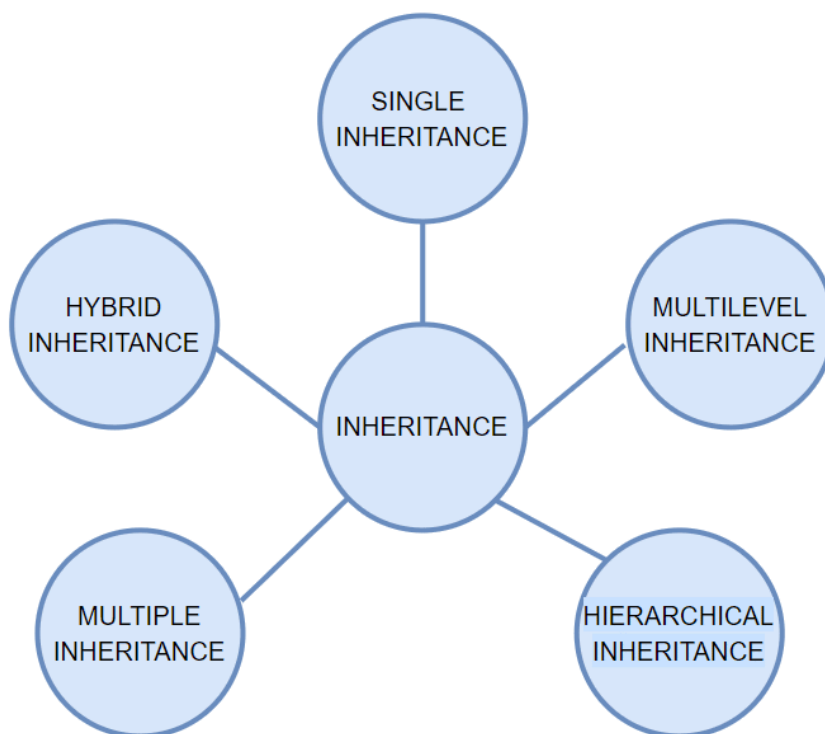
Method overriding is a concept in object-oriented programming where a subclass provides a specific implementation of a method already defined in its superclass. This means that the subclass redefines the behavior of the method inherited from the superclass.

Key points about method overriding:

- Method overriding occurs when a subclass implements a method already defined in its superclass.
- The method signature (name and parameters) in the subclass must match the superclass's method signature.

- Method overriding implements polymorphism, in which a subclass object can be treated as an object of its superclass but with its own specialized behavior.
- Method overriding allows for the customization of behavior in subclasses, providing flexibility and extensibility in object-oriented design.

9. Types of Inheritance in OOPS



- **Single Inheritance:** Child class derived directly from the base class.
- **Multiple Inheritance:** A child class inherits from more than one base class.
- **Multi-level Inheritance:** The child class is derived from a parent class, which is further derived from another class.

- **Multi-path Inheritance:** A child class inherits from two or more classes with a standard base class.
- **Hierarchical Inheritance:** Multiple derived classes are created from a single base class.
- **Hybrid Inheritance:** Combination of two or more types of inheritance.

10. What are the primary characteristics of OOPs?

The main characteristics of OOPs are given as follows:

- In OOP, you combine the code into one unit so you can specify the parameters of each piece of data. This process of wrapping up data into a single unit is called encapsulation.
- By using classes, you can generalise your object types and make your application easier to use. This is termed as an abstraction.
- The ability for a class to inherit characteristics and behaviours from another class allows for more code reuse.
- Polymorphism allows for the creation of several objects from a single, adaptable class of code.

11. Is it possible to call the base class method without creating an instance?

Yes, we can possibly call the base class method without creating an instance in the following 3 cases:

1. If the method is static
2. Calling the inherited method inside a derived class
3. Calling the method using the base keyword from the sub-classes

The most popular case is that of the static methods.

12. What are the limitations of OOPs?

Not Ideal for Small Problems: OOP may be overly complex for small-scale projects, leading to unnecessary overhead.

Requires Extensive Testing: OOP code often requires thorough testing to ensure proper functionality due to its complexity.

Time-Consuming: Developing OOP solutions may take longer due to the need to plan and design object structures.

Necessitates Proper Planning: Effective use of OOP requires careful upfront planning to define classes, relationships, and interactions.

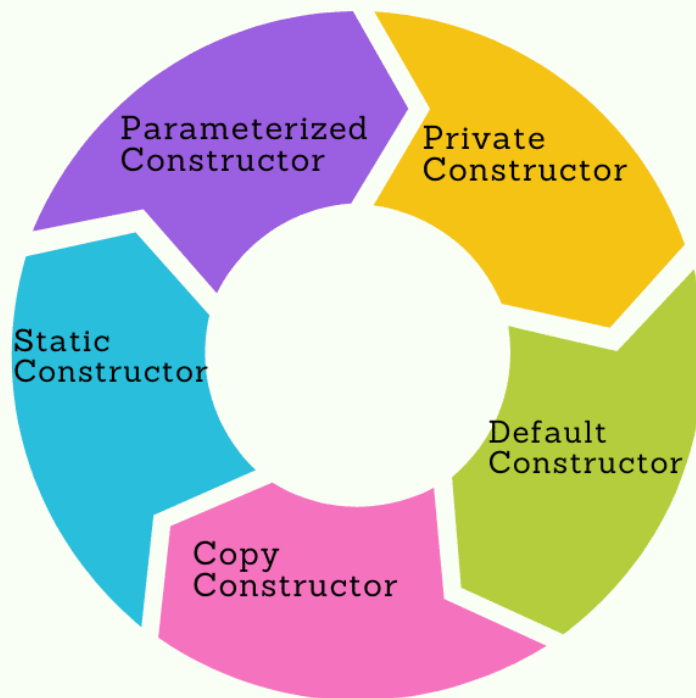
Shift in Thinking: Programmers must adopt a mindset focused on object-oriented problem-solving, which may require a learning curve and mindset shift.

13. What are constructors?

Constructors are unique methods in a class that are automatically called when an instance of the class is created. They are used to initialize the object's state or perform any necessary setup operations.

14. Types of constructor

Types of constructor



Private Constructor: This restricts the instantiation of a class from outside. It is typically used for utility classes with static methods.

Default Constructor: If no constructor is explicitly defined, it is automatically generated by the compiler and initializes instance variables to default values.

Copy Constructor: This function creates a new object by copying the state of an existing object. It is used for creating deep copies or clones.

Static Constructor: Initializes static data members or performs one-time setup tasks for the class, executed only once when the class is loaded.

Parameterized Constructor: Accepts parameters to initialize instance variables during object creation, allowing customization of object state.

15. What is the difference between a class and a structure?

Class: A class is like a blueprint for creating objects in object-oriented programming (OOP). It combines data and behavior into a single unit. Objects made from a class are instances of that class, each with its own data set. Classes are essential for modeling real-world concepts and implementing OOP features like inheritance and polymorphism.

Structure: A structure is a way to group variables of different types under a single name. It's commonly used in languages like C to define custom data types holding multiple elements. Unlike classes, structures don't support methods or inheritance. They're mainly used to organize related data for easier management and manipulation within a program.

16. What are the access modifiers?

Access modifiers are keywords used in object-oriented programming languages to control the visibility and accessibility of classes, methods, and other members within a program.

They determine how these members can be accessed from different parts of the program or outside modules. The primary access modifiers include:

Public: Allows members to be accessed from any part of the program.

Private: Restricts access to members within the same class, preventing access from outside.

Protected: Grants access to members within the same class and its subclasses, but not from outside.

Internal: Limits access to members within the same assembly or module.

Protected Internal: Combines the behavior of protected and internal, allowing access within the same assembly or from subclasses, whether in the same assembly or not.

These access modifiers help enforce encapsulation, control members' visibility and maintain the integrity of the program's design.

17. What languages come under the oops concept?

Simula is known as the first object-oriented programming language, the most popular OOP languages are:

- Java
- JavaScript
- Python
- C++
- Visual Basic
- .NET
- Ruby
- Scala
- PHP

Academy PRO +

Advanced JavaScript Course

Learn advanced JavaScript concepts, including ES6+, async programming, and modern frameworks. Strengthen your skills and build dynamic applications to become a proficient JavaScript developer.

4.5 learning Hrs

2 Projects

Check out the OOPs concept in Python Video.



18. What is inheritance?

Whenever one class is derived from another, it is referred to as inheritance. The child class will inherit all of the parent class's public and protected properties and methods. Apart from the attributes and methods inherited from the parent class, it can also have its own additional set of features. The 'extends' keyword is used to specify an inherited class.

If you derive a class from another class that is known as inheritance. The child class will inherit all the public and protected properties and methods from the parent class. The child class can also have its own properties and methods. An **inherited** class is defined by using the extends keyword.

Multiple inheritance	Multilevel inheritance
If a class inherits more than one base class that time we use multiple inheritance	If a class inherits from another class which itself is a subclass of some other base class then that is known as multilevel inheritance
Example: A class explaining a child. That child class inherits from two base classes which are Mother and Father	Example: A class called as sports bike which is inherited from a base class called bike. And also bike inherits another class Vehicle

19. What is hybrid inheritance?

The type of inheritance formed by the combination of different types of inheritances like single, multiple inheritances, etc. is classified as hybrid inheritance.

20. What is hierarchical inheritance?

In the case of a hierarchical inheritance, multiple subclasses inherit from a parent class. Hierarchical inheritance is a type of inheritance in which multiple classes are descended from a single parent or base class. For example, the fruit class can have 'apple', 'mango', 'banana', 'cherry' etc. as its subclasses.

21. What are the limitations of inheritance?

Inflexibility: Inheritance creates a rigid relationship between classes, making it challenging to change the hierarchy once established.

Complexity: Deep inheritance hierarchies can lead to complex and difficult-to-understand code, especially when multiple levels of inheritance are involved.

Tight Coupling: Subclasses become tightly coupled to their superclasses, increasing dependencies and making modifying or extending the system harder.

Code Reusability Issues: Over-reliance on inheritance for code reuse can lead to duplication and maintenance difficulties.

Violation of Encapsulation: Inherited members are not encapsulated within the subclass, potentially allowing unintended access or modification.

Diamond Problem: In languages supporting multiple inheritance, such as C++, the diamond problem can occur, leading to ambiguity and conflicts in method resolution.

22. What is a superclass?

A superclass is a class from which a subclass or child class is derived. Base class and parent class are other names for a superclass. For example, if Student is a class derived from the Person class, then the Person class will be referred to as the superclass.

```
1  class Person:

2      def __init__(self, name, age):

3          self.name = name

4          self.age = age

5

6      def greet(self):

7          print("Hello, my name is", self.name, "and I
am", self.age, "years old.")

8

9

10 class Student(Person):

11     def __init__(self, name, age, student_id):

12         super().__init__(name, age)

13         self.student_id = student_id

14

15     def study(self):
```

```

1         print(self.name, "is studying.")
4
1
5     # Creating an instance of the Student class
1
6     student = Student("Alice", 20, "12345")
1
7     student.greet() # Output: Hello, my name is Alice and
1
7     I am 20 years old.
1
8     student.study() # Output: Alice is studying.
1
8
1
9
2
0

```

In this example, the **'Person'** class serves as the superclass, providing common attributes like **'name'** and **'age'** and a method **'greet()'**. The **'Student'** class is a subclass derived from **'Person'**, inheriting its properties and methods.

The **'Student'** class also defines its method of **'study()'**. Instances of the **'Student'** class can access both **'Person'** class methods, like **'greet()'**, and its own methods, like **'study()'**.

23. What is a subclass?

A subclass is a class that inherits properties and behaviors from another class, known as its superclass or parent class. It extends or specializes in the functionality of its superclass. For example:

```
1  class Vehicle:

2      def __init__(self, brand):

3          self.brand = brand

4

5      def drive(self):

6          print("Driving a", self.brand)

7

8  class Car(Vehicle):

9      def __init__(self, brand, model):

10         super().__init__(brand)

11         self.model = model

12

13     def honk(self):
```



```
14         print("Honking the horn of", self.brand,  
15               self.model)  
  
16  
17     # Creating an instance of the Car class  
18  
19     my_car = Car("Toyota", "Camry")  
  
20     my_car.drive()    # Output: Driving a Toyota  
  
21     my_car.honk()     # Output: Honking the horn of Toyota  
22                     Camry
```

In this example, the **'Car'** class is a subclass of the **'Vehicle'** class. It inherits the **'drive()'** method from Vehicle and adds its method **'honk()'**. Instances of the Car class have access to both superclass and subclass methods.

Curious about Object-Oriented Programming in Python?

Discover the [Fundamentals of OOP concepts in Python](#) in our blog!

24. What is Polymorphism?

Polymorphism is one of the most used and core concepts in **OOP** languages. It explains the concept of different classes can be used with the same interface. Each of these classes can have its own implementation of the interface.

25. What is static polymorphism?

Static polymorphism, also known as compile-time polymorphism or method overloading, occurs when multiple methods in the same class have the same name but different parameters.

The appropriate execution method is determined at compile time based on the number or type of arguments passed to the process.

26. What is dynamic polymorphism?

Dynamic polymorphism is a method or process that handles a call to an overridden method during runtime rather than at compile time. It is also referred to as dynamic method dispatch or runtime polymorphism. Using method overriding, we can create dynamic polymorphism. An example of runtime polymorphism: is method overriding.

27. What is operator overloading?

[Operator overloading](#) is a feature in object-oriented programming that allows operators to be redefined for custom types. It enables objects to behave like built-in data types with operators such as +, -, *, /, etc.

By overloading operators, programmers can define how operators should work with their custom classes or structures, providing more intuitive and natural syntax for operations involving user-defined types.

Learn more about Java method overloading with examples in our comprehensive guide on [Method Overloading In Java](#).

Also

Discover key concepts and real-world applications in "[Operator Overloading in C++ with examples](#)".

28. Differentiate between overloading and overriding.

When two or more methods in the same class have the same name but different parameters, this is referred to as overloading. The technique of using

the same method signature, i.e., name and parameters, in both the superclass and the child class is known as overriding.

overloading	overriding
If Two or more methods have the same name but they should have different parameters or signature is known as overloading	Child class inherits methods with the same parameters/ signature which are present in the base class is known as overriding
solved during compile-time	solved during runtime

29. What is encapsulation?

Encapsulation in object-oriented programming involves bundling data (attributes or variables) and methods (functions or procedures) that operate on the data into a single unit or class.

This concept allows for data hiding, where the internal state of an object is hidden from the outside world and can only be accessed through designated methods.

This promotes better code organization, security, and maintainability by preventing direct access to sensitive data and enforcing controlled access through well-defined interfaces.

30. What is the difference between public, private and protected access modifiers?

Name	Accessibility from derived class	Accessibility from derived class	Accessibility from any place
Public	Yes	Yes	Yes
Private	Yes	No	No
Protected	Yes	Yes	No

31. What is data abstraction?

Data abstraction is indeed a crucial feature of object-oriented programming. It involves showing only essential information while hiding the implementation details.

For instance, when using a mobile phone, you know how to message or call someone, but you don't need to know the intricate technical processes behind it.

This exemplifies data abstraction as it shields users from unnecessary complexities, focusing on the actions they can perform without burdening them with implementation specifics.

32. How to achieve data abstraction?

Data abstraction can be achieved using two ways:

- Abstract class
- Abstract method

33. What is an abstract class?

An abstract class is also a class which consists of abstract methods.

So what is an abstract method?

These methods are basically declared but not defined and If these methods need to be used later in some subclass that time those methods have to be exclusively defined in the subclass.

34. Differentiate between data abstraction and encapsulation.

abstraction	encapsulation
Abstraction solves the problem at the design level	Encapsulation solves the problem at the implementation level
It helps to hide the implementation details	It wraps the code and data together into a single unit and helps to hides it from the world

35. What are virtual functions?

Virtual functions are declared in a base class that derived classes can override. They allow dynamic polymorphism, enabling selecting the appropriate function implementation at runtime based on the object's type.

In languages like C++, virtual functions are used to achieve late binding or runtime polymorphism.

Explore the principles of object-oriented programming in C++ with our [free OOPs Concepts in C++](#) course.

36. What is a destructor?

A destructor is a unique method in object-oriented programming that is automatically called when an object is destroyed or goes out of scope.

Its primary purpose is to release any resources or memory allocated by the

object during its lifetime, such as closing files, releasing database connections, or deallocating dynamic memory.

This helps in proper memory management and resource cleanup in the program.

37. What is a copy constructor?

By copying the members of an existing object, the copy constructor initialises the members of a newly formed object. The argument for the copy constructor is a reference to an object of the same class. Programmers have the option of directly defining the [copy constructor](#). The compiler defines the copy constructor if the programmer doesn't.

Don't miss out on mastering copy constructors in C++!

Explore our blog on "[Copy Constructor in C++ – Everything You Need To Know](#)" and enhance your expertise.

38. What is the use of 'finalize'?

The finalize() method in object-oriented programming performs cleanup operations on an object before it is garbage collected. It helps release unmanaged resources held by the object, such as file handles or network connections, aiding in memory management.

39. What is Garbage Collection(GC)?

Garbage Collection (GC) is a memory management feature in object-oriented programming languages like C# and Java. It automatically releases memory space reserved for objects no longer in use by the application.

OOP languages with garbage collection utilize GC engines to reclaim memory allocated to objects no longer referenced, ensuring efficient memory usage and preventing memory leaks.

40. What is a final variable?

A final object-oriented programming (OOP) variable can only be initialized once and cannot be reassigned to a different object after its initial assignment. When a variable is marked as final, its reference cannot change, meaning it always refers to the same object in memory.

However, the object's state, such as its data fields, can still be modified. This ensures that the reference to the object remains constant throughout the program, promoting immutability and guaranteeing consistency in object-oriented design.

41. What is an exception?

An exception is a kind of message that interrupts and comes up when there is an issue with the normal execution of a program. Exceptions provide an error and transfer it to the exception handler to resolve it. The state of the program is saved as soon as an exception is raised.

42. What is exception handling?

Exception handling in object-oriented programming is a fundamental concept used to manage errors and exceptional situations that may arise during program execution.

It uses try, catch, and throw keywords to detect, handle, and recover from errors gracefully, preventing program crashes and ensuring robustness.

When an error occurs, an exception object is thrown, and the program flow is redirected to the nearest enclosing catch block that matches the thrown exception type, allowing for appropriate error handling and recovery strategies.

43. What is the difference between an error and an exception?

Error	exception
Errors basically refer the problems. And those problems should not be encountered by applications	Exceptions are basically Conditions that an application might try to catch

44. What is a try/ catch block?

The terms "try" and "catch" describe how to handle exceptions brought on by coding or data mistakes while a program is running. The section of code where exceptions occur is called a try block. Exceptions from try blocks are caught and handled in a catch block.

45. What is a finally block?

Finally designates the section of code that works with the try keyword. It specifies code that is always executed before the method is finished, immediately behind the try and any catch blocks. Regardless of whether an exception is thrown or caught, the finally block is always executed.

46. Can you call the base class method without creating an instance?

Yes, you are allowed to call the base class without instantiating it but there are some conditions that are applicable:

- If it is a static method
- The base class is inherited by some other subclass

47. What is the difference between OOP and SOP?

The critical distinction between Object-Oriented Programming (OOP) and Procedural Programming (also known as Structured Programming) lies in their approach to program organization and design:

Object-Oriented Programming (OOP):

Involves the creation of programs using a collection of objects and their interactions. It emphasizes concepts of objects and classes, where everything is considered as an object with specific properties and behaviors represented in classes. OOP provides encapsulation and abstraction in the code, enabling modular and reusable design. Examples include languages like Java.

Procedural Programming (SOP):

Involves creating programs using a collection of modules or functions. It focuses on functions and structures, where everything is considered functionality and structures, represented using functions. Procedural programming languages, like C, organize code based on procedures or routines, with less emphasis on data and its organization within objects.

48. What is the difference between a class and an object?

In object-oriented programming (OOP), the key difference between a class and an object lies in their roles and definitions:

Class:

A class is a blueprint or template for creating objects. It defines the properties (attributes) and behaviors (methods) that all objects of that class will have. Classes serve as the foundation for object creation and encapsulate the common characteristics shared by multiple objects.

Object:

An object is an instance of a class. It represents a specific entity or instance in memory that conforms to the structure defined by its class. Objects have their unique state (attribute values) and behavior (method executions) based on the class from which they were instantiated.

```
1  class Car:
2
3      def __init__(self, brand, model):
4
5          self.brand = brand
6
7          self.model = model
8
9      def drive(self):
10
11          print("Driving", self.brand, self.model)
12
13
14  # Creating an object (instance) of the Car class
15
16  my_car = Car("Toyota", "Camry")
17
18
19  # Accessing attributes and invoking methods of the
20  # object
21
22  print(my_car.brand)    # Output: Toyota
23
24  print(my_car.model)    # Output: Camry
```

```
my_car.drive()           # Output: Driving Toyota Camry
```

In this example, **'Car'** is a class that defines the blueprint for creating cars. **'my_car'** is an object (instance) of the **'Car'** class, representing a specific car entity with its brand and model attributes. The object can access methods like **drive()** defined in its class.

49. What are 'access specifiers'?

In object-oriented programming, access specifiers are keywords used to control the visibility and accessibility of class members, such as attributes and methods. They specify the level of access that other classes or code modules have to these members.

In most OOP languages, the primary access specifiers include:

Public: Allows unrestricted access to class members from anywhere in the program.

Private: This option restricts access to class members only within the class where they are declared, preventing access from outside.

Protected: Allows access to class members within the class where they are declared and in its subclasses, but not from outside.

These access specifiers help enforce encapsulation, define the boundaries of class members, and maintain the integrity of the program's design.

50. Can you create an instance of an abstract class?

No, an instance of the Abstract class cannot be created. To implement the abstract Class, abstract methods, the Abstract Class should be extended by another class, and the object of the implementation class can be created.

OOPs Interview Questions for Experienced

51. What is an interface?

An interface is a user-defined [data type](#) and is a collection of abstract methods. A class implements an interface, thereby inheriting the abstract methods of the interface. A class describes an object's attributes and behaviours, and an interface contains behaviours that a class implements. The Class represents "how," and the interface represents "what".

52. What are pure virtual functions?

A pure virtual function/method is a function whose implementations are not provided in the base class, and only a declaration is provided. The pure virtual function can have its implementation code in the derived class; otherwise, the derived class will also be considered an abstract Class. The Class containing pure virtual functions is abstract.

53. Differentiate between a class and a method.

A class is a blueprint of objects, and it consists of the properties and behaviour of the objects.

Methods are programming constructs that perform specific tasks/behaviour.

54. Differentiate between an abstract class and an interface?

An interface can have only abstract methods, but an Abstract class can have abstract and non-abstract methods.

The interface should be used if just the requirement specification is known and nothing about implementation. If the implementation is known, but partially, then an abstract class should be used. If the implementation is known completely, then a concrete Class should be used.

55. What is the virtual function?

A virtual function is declared within a base class that can be overridden in derived classes. This enables polymorphic behavior, where the appropriate function implementation is selected based on the object type during runtime.

56. What are the characteristics of an abstract class?

1. A class having at least one pure virtual function is called an Abstract class.
2. An Abstract class cannot have objects created, i.e., an abstract class cannot be instantiated, but Object references can be created.
3. An Abstract class can have non-abstract functions and pure virtual functions also.
4. The pure virtual function can have its implementation code in the derived class; otherwise, the derived class will also be considered an abstract Class

57. What is constructor chaining?

Constructor chaining is a method to call one constructor from another concerning a current object reference. It can be done in two ways: -

1. Using the "this" keyword, the reference can be made to the constructor in the current class.
2. To call the constructor from the base class "super" keyword will be used.

58. What is Coupling in OOP, and why is it helpful?

The degree of dependency between the components is called coupling.

Types of Coupling

A. **Tight Coupling** – If the dependency between components is high, these components are called tightly coupled.

Ex: - Below three Classes are highly dependent on each other hence they are tightly coupled.

```
1  class P
2  {
3  static int a = Q.j;
4  }
5
6  class Q
7  {
8  static int j = R.method();
9  }
10
11 class R
12 {
```

```
13 public static int method() {  
  
14     return 3;  
  
15 }
```

B. Loose Coupling – If the dependency between components is low, it is called loose coupling. Loose coupling is preferred because of the following reasons:-

1. It increases the maintainability of code
2. It provides reusability of code

59. Name the operators that cannot be overloaded

The operators that cannot be overloaded typically include:

. (dot) member selection operator

.* member selection with a pointer to the object operator (in C++)

:: scope resolution operator

?: ternary operator

Additionally, in languages like Java, operators such as &&, ||, and ?: cannot be overloaded.

The + operator can be overloaded in many programming languages, such as C++ and C#.

60. What is Cohesion in OOP?

Cohesion in object-oriented programming refers to the degree to which elements within a module (such as a class) are related and serve a common purpose. Higher cohesion indicates that the elements within the module are closely related and focused on a specific task or functionality.

Advantages of high cohesion include improved code maintainability and reuse because modules with well-defined and specific functionality are easier to understand, modify, and reuse in different parts of the program or in other programs.

61. What are the levels of data abstraction?

Levels of data abstraction refer to the different layers of abstraction within a software system, ranging from low-level implementation details to high-level conceptual models.

Abstraction is achieved through abstract classes, interfaces, and encapsulation, which hide complexity and provide simplified interfaces for interacting with data and functionality.

These levels of abstraction help manage complexity, improve maintainability, and facilitate modular design in software development.

62. What are the types of variables in OOP?

Variables are basic units to store data in RAM for Java programs.

Variables should be declared before using them in Java programming. Variable initialization can be static or dynamic. The syntax for variable declaration and static initialization is: -

Types of variables

- **Primitive Variables:** It is used to represent primitive values like int, float, etc.

- **Reference Variables:** It is used to refer to objects in Java.
- **Instance Variables:** Variables whose value varied from object to object are instance variables. For every object, a separate copy of the instance variable is created. Instance variables are declared within the Class and outside any method/block/constructor
- **Static variables:** For static Variables, a single copy of the variable is created, and that copy is shared between every Class object. The static variable is created during class loading and destroyed at class unloading.
- Static variables can be accessed directly from the static and instance area. We are not required to perform initialization explicitly for static variables, and JVM will provide default values.
- **Local Variables:** Variables declared inside a method or block or constructor are local variables. Hence the scope of local variables is the same as the block's scope in which we declared that variable.

JVM doesn't provide default values, and before using that variable, the initialization should be performed explicitly.

63. What do you understand by Garbage Collection in the OOPs world?

Garbage collection is a memory recovery technique included in programming languages like C# and Java. A GC-enabled programming language contains one or more garbage collectors that automatically free up memory space allocated to objects that are no longer needed by the program.

64. Is it possible to run a Java application without implementing the OOPs concept?

No, since Java programmes are founded on the concept of object-oriented programming models, or OOPs, a Java application cannot be implemented without it.

Understand the fundamentals of Object-Oriented Programming in Java!
Our blog, "[OOps Concept in Java with Examples](#)," offers real-world examples and practical insights.

65. What is the output of the below code?

```
1  class Person
2  {
3      private String show()
4      {
5          return "This is a person";
6      }
7  }
8  class Teacher extends Person
9  {
10     protected String show()
11     {
12         return "This is a teacher";
```

```
13  }

14  }

15  public class MathsTeacher extends Person

16  {

17      @Override      public final String show()

18  {

19      return "This is a Maths teacher";

20  }

21  public static void main(String[] name)

22  {

23      final Person mt = new MathsTeacher();

24      System.out.print(mt.show());

25  }

26  }
```

1 The output will be: This is a Maths teacher

66. Find the output of the below code.

```
1  class Arithmetic
2  {
3  public final double var = 5;
4  }
5  class DeepArith extends Arithmetic
6  {
7  public final double var = 10;
8  }
9  public class AdvancedArith extends DeepArith
10 {
11 public final double secret = 20;
12 public static void main(String[] num)
13 {
```

```
14 Arithmetic arith = new AdvancedArith();

15 System.out.print(arith.var);

16 }

17 }
```

1 The correct output **for this** code is 5.

67. Predict the output of the following.

```
1 class Parent

2 {

3     public void display()

4     {

5         System.out.println("Parent");

6     }

7 }

8 class Child extends Parent
```

```
9  { private void display()

10 { System.out.println("Child");

11 }

12 }

13 public class main

14 {

15 public static void main(String args[])

16 {

17 Parent node = new Child(); node.show();

18 }

19 }
```

Running **this** code will generate a compile error as a
1 sub-**class** function overriding a **super class** function
cannot be given more restrictive access.

68. Implement a Singleton class in Java.

```
1  public class Singleton {
2      private static Singleton instance;
3
4      private Singleton() {
5          // Private constructor to prevent
6          instantiation.
7      }
8
9      public static Singleton getInstance() {
10         if (instance == null) {
11             synchronized (Singleton.class) {
12                 if (instance == null) {
13                     instance = new Singleton();
14                 }
15             }
16         }
17     }
18 }
```

```
16         }

17         return instance;

18     }

    }
```

69. Implement a Stack data structure using a LinkedList in Python.

```
1  class Node:

2      def __init__(self, data=None):

3          self.data = data

4          self.next = None

5

6  class Stack:

7      def __init__(self):

8          self.head = None

9

10     def is_empty(self):
```



```
11         return self.head is None

12

13     def push(self, data):

14         new_node = Node(data)

15         new_node.next = self.head

16         self.head = new_node

17

18     def pop(self):

19         if self.is_empty():

20             raise Exception("Stack is empty")

21         popped_value = self.head.data

22         self.head = self.head.next

23         return popped_value
```

70. Implement an abstract class in C# with an abstract method.

```
1  abstract class Shape

2  {

3      public abstract double CalculateArea();

4  }

5

6  class Circle : Shape

7  {

8      private double radius;

9

10     public Circle(double radius)

11     {

12         this.radius = radius;

13     }

14

15     public override double CalculateArea()
```

```
16      {  
  
17          return Math.PI * radius * radius;  
  
18      }  
  
19 }
```

Conclusion

This compilation of top OOPs interview questions and answers will be valuable for your upcoming interviews or viva sessions. Mastering object-oriented programming is gradual, and practical application is crucial in solidifying your understanding. To further enhance your skills, explore [free online courses](#) for object-oriented programming that can help deepen your knowledge and improve your practical application of these concepts.

To delve deeper into the real-world applications of OOPs, consider enrolling in courses like the [Free Python Fundamentals Course for Beginners](#) and the [Free Java Programming Course](#) with Certificate offered by [Great Learning](#).

These courses provide hands-on experience and insights into applying OOP principles, helping you to strengthen your skills and prepare you for various job interviews and practical scenarios.



Frequently Asked OOPs Interview Questions

Q: What are the 4 basics of OOP?

OOP stands for Object-Oriented Programming, and its four basic principles are Encapsulation, Abstraction, Polymorphism, and Inheritance. OOP enables programmers to consider software development as if they are working with actual entities. In OOP, some objects have a field where data/knowledge can be stored and can do several methods.

Q: What is the object-oriented programming interview?

Object-Oriented Programming, also usually called OOPS, is a kind of programming that is more object-based and not just based on functions or procedures. Individual objects are collected into several classes. Real-world entities such as inheritance, polymorphism, and hiding are implemented by OOPS into programming. It also enables binding data as well as code together.

Q: What are the 3 principles of OOP?

The three main principles of Object-Oriented Programming are Encapsulation, inheritance, and polymorphism.

Q: What is the concept of OOPS?

OOPS or Object-Oriented Programming System is a programming concept that mainly works based on Encapsulation, Abstraction, Polymorphism, and Inheritance. The usual concept of OOPs is to create objects, use them again all through the program, and finally manipulate these objects to fetch our results.

Q: Why is OOPS used?

The main aim of an Object-Oriented Programming System is to implement real-world entities such as polymorphism, inheritance, hiding, and many more in programming. The aim lies in binding together the data as well as functions that work on them so that other parts of the code cannot access the data other than that function.

Q: What is polymorphism in OOPS?

Polymorphism in an Object-Oriented Programming System is a feature of object-based programming languages that enable a particular routine to use variables of several types at different times. It can also be called the ability of a programming language to present the same interface for different primary data types.

Q: Who is the father of OOPS?

The father of the Object-Oriented Programming System is considered to be Alan Kay by some people. He identified some characteristics as basics to OOP Kay 1993:1. He coined OOPs around 1966 or 1967 when he was at grad school.

Q: What are the main features of OOPS?

Some of the main features in OOPS include Classes, Objects, Data Abstraction, Encapsulation, Inheritance, and Polymorphism. OOP is a programming paradigm that is based on the idea of objects.

Q: What are the advantages of OOPS?

Since OOP is one of the main development approaches which is easily accepted, the advantages are many. Some of the advantages of OOPS include Reusability, Data Redundancy, Code Maintenance, Security, Design Benefits, Easy Troubleshooting, Better Productivity, Polymorphism Flexibility, and Problem-solving.

If you wish to learn more about such concepts, you can join a [Software Engineering courses](#) that will help you upskill.

Landing a job that requires SQL skills often means preparing for a challenging interview. SQL is a foundational technology for database management, so employers test candidates on various concepts ranging from basic to advanced-level queries. This blog covers common SQL interview questions and their answers to help you succeed in your next interview.

What is SQL?

SQL or Structured Query Language, is a programming language designed for managing and querying relational databases. It allows you to create, read, update, and delete data in databases, making it essential for working with data-driven systems. It forms the backbone of most modern applications, making it a must-have skill for developers, analysts, and data professionals alike.

Academy Pro

[SQL Course](#)

Master SQL and Database management with this SQL course: Practical training with guided projects, AI support, and expert instructors.

7 Hrs

2 Projects

Basic SQL Questions

1. What are the types of SQL commands?

SQL commands are grouped into the following categories:

- **DDL (Data Definition Language):** Commands like CREATE, ALTER and DROP that define or modify database structure. For example, creating a new table in a database involves a CREATE command.
- **DML (Data Manipulation Language):** Commands like INSERT, UPDATE and DELETE for modifying data. These commands interact directly with the rows of a table.
- **DQL (Data Query Language):** The SELECT statement used to query data. This command helps retrieve specific information from tables.
- **DCL (Data Control Language):** Commands like GRANT, REVOKE to control access to the database. They ensure data security.
- **TCL (Transaction Control Language):** Commands like COMMIT, ROLLBACK for managing transactions, ensuring data consistency.

2. What is a primary key?

A **primary key** is a unique identifier for a record in a table. It ensures that each row in the table is unique and cannot contain NULL values. It is the cornerstone of relational integrity. For example:

```
1 CREATE TABLE Employees (  
2     EmployeeID INT PRIMARY KEY,  
3     Name VARCHAR(50),  
4     Department VARCHAR(50)  
5 );
```

3. What is the difference between WHERE and HAVING?

- **WHERE:** Filters rows before grouping data. It is used for raw data filtration.
- **HAVING:** Filters groups after the GROUP BY clause, making it useful for aggregated data. Example:

```
1 SELECT Department, COUNT(*)  
2  
3 FROM Employees  
4
```



```
5 WHERE Department IS NOT NULL
```

```
6
```

```
7 GROUP BY Department
```

```
8
```

```
9 HAVING COUNT(*) > 5;
```

4. What is a foreign key?

A **foreign key** is a field in one table that links to the primary key in another table. It enforces referential integrity between two tables, ensuring valid relationships. Example:

```
1 CREATE TABLE Orders (
```

```
2
```

```
3     OrderID INT PRIMARY KEY,
```

```
4
```

```
5     CustomerID INT,
```

```
6
```

```
7     FOREIGN KEY (CustomerID) REFERENCES  
Customers (CustomerID)
```

8

9);

5. What are indexes in SQL?

Indexes are special lookup tables used to speed up data retrieval. They significantly improve query performance by allowing the database to find data without scanning every row in a table. However, they may slow down INSERT or UPDATE operations due to the maintenance of index data. Example:

```
1 CREATE INDEX idx_customer_name
```

```
2
```

```
3 ON Customers (CustomerName);
```

6. Explain the difference between DELETE and TRUNCATE.

- **DELETE:** Removes specific rows based on a condition and can use a WHERE clause. It logs each row deleted.
- **TRUNCATE:** Removes all rows from a table without logging individual row deletions, making it faster but less flexible. It resets table identity values.

7. What is the use of the GROUP BY clause?

The **GROUP BY** clause is used to group rows with the same values in specified columns and perform aggregate functions like COUNT, SUM, or AVG on them. Example:

```
1  SELECT Department, AVG(Salary)
```

```
2
```

```
3  FROM Employees
```

```
4
```

```
5  GROUP BY Department;
```

8. How do you fetch unique records?

Use the DISTINCT keyword to fetch unique records. This ensures no duplicate rows are returned. Example:

```
1  SELECT DISTINCT Department
```

```
2  FROM Employees;
```

9. What is a default constraint in SQL?

A **default constraint** sets a default value for a column if no value is specified during insertion. Example:

```
1  CREATE TABLE Products (
```

```
2
```

```
3      ProductID INT,
```

4

5 Price **DECIMAL**(10, 2) **DEFAULT** 0.00

6

7);

10. Explain LIKE in SQL.

The **LIKE** operator is used for pattern matching in a WHERE clause. It is particularly useful for flexible searches. Example:

1 **SELECT** *

2

3 **FROM** Employees

4

5 **WHERE** Name **LIKE** 'A%';

This query retrieves all employees whose names start with 'A'.

Intermediate SQL interview questions with detailed answers:

11. Find Customer Order Counts

Question:

Write a query to list customers who have placed more than 5 orders. Use the customers and orders tables.

Answer:

```
1  SELECT c.customer_id, c.customer_name,  
   COUNT(o.order_id) AS order_count  
  
2  
  
3  JOIN orders o ON c.customer_id = o.customer_id  
  
4  
  
5  GROUP BY c.customer_id, c.customer_name  
  
6  
  
7  HAVING COUNT(o.order_id) > 5;
```

- **Explanation:** This query joins the customers and orders tables on customer_id. It groups by customer_id and customer_name, then counts the number of orders for each customer. The HAVING clause filters the results to show customers with more than 5 orders.

12. Aggregate Functions with GROUP BY

Question:

Calculate the total revenue, average revenue, and the count of orders grouped by product category in the sales table.

Answer:

```
1  SELECT product_category,  
2  
3      SUM(revenue) AS total_revenue,  
4  
5      AVG(revenue) AS average_revenue,  
6  
7      COUNT(order_id) AS order_count  
8  
9  FROM sales  
10  
11 GROUP BY product_category;
```

- **Explanation:** This query uses SUM() to calculate total revenue, AVG() to find the average revenue, and COUNT() to count the number of orders. The GROUP BY clause groups the result by product category.

13. INNER JOIN Practice

Question:

Join the employees and departments tables. Return the employee name and department name for all employees.

Answer:

```
1  SELECT e.employee_name, d.department_name
2
3  FROM employees e
4
5  INNER JOIN departments d ON e.department_id =
   d.department_id;
```

- **Explanation:** An INNER JOIN is used to combine rows from employees and departments where the department_id matches in both tables. This returns the employee names along with their respective department names.

14. Subquery for Average Salary

Question:

Write a query to find the names of employees who earn more than the average salary in the company.

Answer:

```
1  SELECT employee_name
2
3  FROM employees
4
5  WHERE salary > (SELECT AVG(salary) FROM employees);
```

- **Explanation:** The subquery (SELECT AVG(salary) FROM employees) calculates the average salary. The outer query returns employees whose salary is greater than this average.

15. LEFT JOIN to Include Nulls**Question:**

Write a query to list all employees and their department names, including employees who don't belong to any department.

Answer:


```
1  SELECT e.employee_name, d.department_name
2
3  FROM employees e
4
5  LEFT JOIN departments d ON e.department_id =
   d.department_id;
```

- **Explanation:** A LEFT JOIN is used here to include all employees. If an employee doesn't belong to a department, the result will show NULL for the department name.

16. Self Join to Find Manager-Employee Relationship

Question:

Write a query to list employees along with their manager's name. Assume there's a column `manager_id` in the `employees` table that points to another employee's `employee_id`.

Answer:

```
1  SELECT e.employee_name AS employee, m.employee_name AS
2  manager
3
```

4 **FROM** employees e

5

LEFT JOIN employees m **ON** e.manager_id = m.employee_id;

- **Explanation:** This is a self join on the employees table, where the manager_id in the employee record matches the employee_id in the manager's record. It returns the employee names along with their manager's names.

17. Using UNION to Combine Results

Question:

Write a query to list all employees and all customers (from the employees and customers tables) who have made orders. Ensure no duplicate records are returned.

Answer:

1 **SELECT** employee_name **AS** name **FROM** employees

2

3 **UNION**

4

5 **SELECT** customer_name **AS** name **FROM** customers

6

```
7 WHERE customer_id IN (SELECT DISTINCT customer_id FROM  
orders);
```

- **Explanation:** The UNION operator combines results from the employees and customers tables. The WHERE clause filters customers who have placed orders, ensuring unique names across both datasets.

18. Using CASE for Conditional Logic

Question:

Write a query to categorize employees based on their salary into "High", "Medium", and "Low" categories.

Answer:

```
1 SELECT employee_name,  
2  
3 CASE  
4  
5 WHEN salary > 100000 THEN 'High'  
6
```

```
7          WHEN salary BETWEEN 50000 AND 100000 THEN
      'Medium'
8
9
      ELSE 'Low'
10
11
      END AS salary_category
12
13
FROM employees;
```

- **Explanation:** The CASE statement is used to categorize employees based on their salary. The result categorizes them as "High", "Medium", or "Low".

19. Find Duplicate Records

Question:

Write a query to find duplicate email addresses from the users table. Assume the users table has a column email.

Answer:

```
1 SELECT email, COUNT(*) AS count
```

```
2 FROM users

3 GROUP BY email

4 HAVING COUNT(*) > 1;
```

- **Explanation:** This query groups records by email and counts how many times each email appears. The HAVING clause filters for emails that appear more than once.

20. Using Window Functions (ROW_NUMBER)

Question:

Write a query to assign a row number to each employee, ordered by their salary in descending order.

Answer:

```
1 SELECT employee_name, salary,
2
3         ROW_NUMBER() OVER (ORDER BY salary DESC) AS
   row_num
4
5 FROM employees;
```

- **Explanation:** The ROW_NUMBER() window function assigns a unique row number to each row, ordered by salary in descending order. It doesn't require a GROUP BY clause because it is a window function operating over the entire result set.

Advanced SQL interview questions

Here are **15 advanced SQL interview questions or concepts** with **detailed answers**, designed to test in-depth SQL knowledge and problem-solving ability:

21. What are Window Functions? Explain with an example.

Answer:

Window functions perform calculations across a set of table rows that are related to the current row. Unlike regular aggregate functions, which return a single value for a group of rows, window functions return a value for each row in the result set.

Example:

```
1  SELECT employee_name, salary,
2
3         RANK() OVER (ORDER BY salary DESC) AS rank
4
5  FROM employees;
```

- **Explanation:**
 - **RANK()** is a window function that assigns a unique rank to each row within a result set, based on the salary column. The window is defined by the ORDER BY clause, and the rank restarts when there are ties in salary.

22. What is the difference between INNER JOIN, LEFT JOIN, and RIGHT JOIN?

Answer:

- **INNER JOIN** returns only the rows that have matching values in both tables.
- **LEFT JOIN** returns all rows from the left table and the matched rows from the right table. If there is no match, the result will contain NULL for columns from the right table.
- **RIGHT JOIN** is similar to LEFT JOIN, but it returns all rows from the right table, and the matching rows from the left table. If there is no match, the result will contain NULL for columns from the left table.

23. How would you find the Nth highest salary in a table?

Answer:

To find the Nth highest salary, you can use ROW_NUMBER() or RANK() along with a subquery or a CTE.

Example for Nth highest salary:

```
1  WITH RankedSalaries AS (  
2  
3      SELECT salary, RANK() OVER (ORDER BY salary DESC)  
4  AS rank  
5  
6  
7  FROM employees  
8  
9  
10  
11  SELECT salary  
12  
13  FROM RankedSalaries  
14  
15  WHERE rank = 3;  -- Nth value, here it is the 3rd  
16  highest salary
```


- **Explanation:**

The RANK() function ranks salaries in descending order. By filtering for rank = 3, you get the 3rd highest salary.

24. Explain the concept of Normalization and its different normal forms.

Answer:

Normalization is the process of organizing a database to reduce redundancy and dependency by splitting large tables into smaller, related tables. The goal is to eliminate undesirable characteristics like insertion, update, and deletion anomalies.

The common **normal forms (NF)** are:

- **1NF (First Normal Form):** Ensures that each column contains atomic (indivisible) values and each row is unique.
- **2NF (Second Normal Form):** Meets 1NF requirements and ensures that all non-key columns are fully dependent on the primary key.
- **3NF (Third Normal Form):** Meets 2NF requirements and ensures that there are no transitive dependencies (i.e., non-key columns depend on other non-key columns).
- **BCNF (Boyce-Codd Normal Form):** A stricter version of 3NF, where every determinant is a candidate key.

Normalization helps in reducing data redundancy but might increase the complexity of queries, so in some cases, **denormalization** is used for performance reasons.

25. What is a Common Table Expression (CTE), and how is it different from a subquery?

Answer:

A **CTE** is a temporary result set that can be referenced within a SELECT,

INSERT, UPDATE, or DELETE statement. It is defined using the WITH keyword before the main query.

Example:

```
1  WITH EmployeeCTE AS (  
2  
3      SELECT employee_id, salary  
4  
5      FROM employees  
6  
7      WHERE department_id = 10  
8  
9  )  
10  
11 SELECT * FROM EmployeeCTE;
```

- **Explanation:**

A CTE is similar to a subquery but can be more readable and

reusable. It allows recursion and can be referenced multiple times in the main query.

26. What are Indexes and how do they improve query performance?

Answer:

An **index** is a database object that speeds up data retrieval operations on a table. Indexes are created on one or more columns and help the database engine quickly locate rows without scanning the entire table.

- **How they improve performance:**

Indexes reduce query execution time by allowing faster access to rows, especially for queries with WHERE, JOIN, or ORDER BY clauses. However, indexes can slow down INSERT, UPDATE, and DELETE operations because the index also needs to be updated.

- **Types of indexes:**

- **B-tree index:** Default index type, useful for range queries and equality searches.
- **Hash index:** Used for equality comparisons.
- **Full-text index:** Used for searching large text fields.

27. What is the difference between TRUNCATE and DELETE?

Answer:

- **DELETE:**

- Removes rows from a table based on a condition.
- Slower because it logs each row deletion and maintains referential integrity (with foreign keys).
- Can be rolled back if wrapped in a transaction.

- **TRUNCATE:**

- Removes all rows from a table, without logging each row individually.
- Faster than DELETE because it doesn't maintain transaction logs for each row.
- Cannot be rolled back (in most cases) and does not fire triggers.

28. What are Triggers in SQL?

Answer:

A **trigger** is a set of SQL statements that automatically execute or "fire" when certain events (INSERT, UPDATE, DELETE) occur on a table. Triggers are used for enforcing business rules, data validation, and auditing.

- **Example:** A trigger can be set to track whenever a row is inserted into the employees table, automatically logging the action in an audit table.

29. What are Transaction and ACID properties?

Answer:

A **transaction** is a sequence of one or more SQL operations that are executed as a single unit of work. Transactions ensure data integrity, even in the event of a failure.

ACID Properties:

- **Atomicity:** Ensures that all operations in a transaction are completed successfully or none are.
- **Consistency:** Ensures the database moves from one valid state to another.
- **Isolation:** Ensures that operations of one transaction are not visible to others until the transaction is complete.

- **Durability:** Ensures that once a transaction is committed, its changes are permanent.

30. How would you write a query to find employees who worked in multiple departments?

Answer:

To find employees who worked in multiple departments, you can group by employee_id and count distinct department_id.

```
1  SELECT employee_id, COUNT(DISTINCT department_id) AS
   department_count
2
3  FROM employee_department
4
5  GROUP BY employee_id
6
7  HAVING COUNT(DISTINCT department_id) > 1;
```

- **Explanation:**

This query counts the number of distinct departments an employee has worked in, and filters those employees who have worked in more than one department.

31. What is a Subquery and how does it differ from a Join?

Answer:

A **subquery** is a query embedded within another query. It can be used to filter results or calculate values for comparison. A subquery is executed first, and its results are used by the outer query.

- **Difference from a JOIN:**
 - **Subquery:** Runs a separate query and provides results to the main query.
 - **JOIN:** Combines rows from multiple tables based on a related column and executes in one step.

32. Explain the difference between GROUP BY and DISTINCT.**Answer:**

- **GROUP BY:**
 - Used to aggregate rows into groups based on one or more columns. It is often used with aggregate functions (like COUNT(), SUM(), AVG(), etc.).
- **DISTINCT:**
 - Used to remove duplicate rows from the result set, returning only unique values.

Example for GROUP BY:

```
1  SELECT department_id, COUNT(*) FROM employees GROUP BY  
   department_id;
```

Example for DISTINCT:

```
1 SELECT DISTINCT department_id FROM employees;
```

33. What is Denormalization?

Answer:

Denormalization is the process of combining tables or adding redundant data to a database to improve read performance. It reduces the need for complex joins by storing the same data in multiple places, which can speed up queries at the cost of additional storage and potential data inconsistency.

34. How can you improve the performance of a slow query?

Answer:

To improve query performance:

- **Use indexes** on frequently queried columns.
- **Optimize joins**: Avoid unnecessary joins and subqueries.
- **Rewrite subqueries** as joins where possible.
- **Limit data**: Only select the necessary columns, not `SELECT *`.
- **Use EXPLAIN** to analyze query plans and identify bottlenecks.
- **Avoid complex operations** in WHERE clauses (e.g., functions on columns).

35. What is Cascading in Foreign Keys?

Answer:

Cascading is a set of rules that define what happens to related rows when a row in a referenced table is updated or deleted.

- **CASCADE**: Automatically updates or deletes rows in the child table when the corresponding row in the parent table is updated or deleted.
- **SET NULL**: Sets the foreign key values to NULL when the parent row is deleted.

- **NO ACTION:** Prevents the update or delete operation if there are any related rows in the child table.

These questions and concepts cover a broad range of advanced SQL topics such as window functions, joins, normalization, subqueries, and transaction management. Mastery of these areas will demonstrate a solid understanding of relational database systems and SQL performance optimization.

Tips for SQL Interviews

Here are **5-7 quick tips** for succeeding in an **SQL interview**:

1. **Master SQL Basics:** Understand core concepts like SELECT, JOIN, GROUP BY, and aggregation functions.
2. **Practice Regularly:** Solve SQL problems on platforms like LeetCode, HackerRank, or SQLZoo to sharpen your skills.
3. **Optimize Queries:** Focus on writing efficient queries by using proper indexing and understanding query plans.
4. **Know Advanced Concepts:** Be prepared to discuss window functions, CTEs, normalization, and transaction management.
5. **Think Aloud:** During the interview, explain your thought process clearly when solving a problem.
6. **Consider Edge Cases:** Handle NULL values, empty sets, and duplicates in your queries to ensure robustness.
7. **Stay Calm:** Don't panic if you don't know the answer immediately—focus on problem-solving steps and communicating your approach.

These tips will help you prepare effectively and confidently tackle SQL interview challenges!

Enhance Your SQL Skills with Free Courses

Ready to take your SQL skills to the next level?

If you're looking for structured learning to boost your SQL knowledge, check out [free SQL courses](#) to kickstart your learning.

Conclusion

In SQL interviews, success comes from a combination of mastering core concepts, consistent practice, and a calm, structured approach to solving problems. By understanding key SQL operations, optimizing your queries, and being prepared to tackle complex scenarios, you'll be able to impress interviewers with both your technical skills and your problem-solving mindset.

As you continue to refine your skills and knowledge, remember that consistent learning and practice are essential to staying ahead in the field. If you're ready to dive deeper and enhance your SQL expertise, consider exploring [free SQL courses](#) by Great Learning. These courses are designed to help you understand both basic and advanced concepts with hands-on experience, perfect for boosting your confidence in SQL interviews.

Do you have a job interview coming up? Wouldn't it be great if you knew exactly what questions a hiring manager would be asking you? The best way to get ready for an interview is to take the time to review the different types of interview questions you will most likely be asked.

Here is a list of more top 25 most common interview questions to help you [crack a job interview](#).

What are the 25 most common interview questions and answers?

Here are a few of the most common interview questions and answers

1. [Tell me about yourself](#)
2. [What are your weakness](#)
3. [Why should we hire you?](#)

4. Where do you see yourself in five years?
5. What causes you to get motivated?
6. What are your great strengths?
7. Do you have any questions?
8. Why do you want to figure at this company?
9. How do you handle stress?
10. What Are You Passionate About?
11. Are You Willing to Relocate?
12. How did you get to know about this position?
13. Why do you want this job role?
14. What can you bring to this organization?
15. Tell me about your most outstanding professional achievement
16. Tell me about a conflict you've faced at the workplace and how you dealt with it
17. Tell me about a situation/scenario where you demonstrated leadership skills
18. Tell me about a time you disagreed with a decision at work/workplace
19. Tell me about your failure
20. Why are you leaving your current job?
21. Why do you have a gap in your career?
22. Tell me about something other than your resume
23. What are your salary expectations for this job role?
24. How do you think we can improve as a company?
25. Any questions for us?

1. Tell me about yourself

One of the basic interview questions, almost every interviewer starts with this question to grasp more about you, but mostly because they determine whether the candidate can introduce himself and needs to grasp that candidate personally.

Your answer should be about two or three minutes long and explain what causes you to be unique from others for that job role. Also cover your education details, your areas of interest, your previous job, and work experience.

2. What are your biggest weaknesses?

As a candidate, you need to know that everybody has weaknesses and be an honest person, explain how you overcome your weakness. The interviewer just wants to understand about weakness and the way you overcame that weakness. While answering this question, always take one in all your weaknesses and switch it into one among your strengths.

Consider this scenario, if a candidate has a fear to present himself before a crowd he can answer like this- “ Sir, I had facing problem presenting myself in front of the crowd during my initial college days. But after joining automation clubs, where we've got to manage juniors and address them to perform a specific task. So now I'm confident to present myself in front of a crowd”



3. Why should we hire you?

As probably there are plenty of scholars who are already applied for that job. So you have got to clarify what causes you to differ from others, how passionate that job role and skills fit their requirements.

Sample answer:

Well, as a start, I've got all the talent and work experience required for the job. I've worked as a Sales Manager for 3+ years. I've got worked on numerous client projects. On top of that, I've got experience working with tech companies, so I'll be ready to develop all the product specifics much faster than the opposite candidates.

4. Where do you see yourself in the next five years?

Well, here it comes! One of the most common interview questions asked by interviewers. It would help if you were honest and practical here. The interviewer or the hiring manager here wants to know about the following:

- your expectations from your career
- your ambition or purpose
- if you are on the right track or in sync considering the job role you are interviewing for

As much as it is ok to say you are unsure about how the future unfolds, you must ensure your goals post five years are in sync with the job role you have applied for. Do not speak about your dream to work at some big company like Google or any other company. Keep it simple and just about your career.

Types of Interview Questions

5. What motivates you?

The interviewer asks this question to check your level of self-awareness and wants to know about the sources of motivation that align with the role and company. The answer should be as specific as possible, always provide real-life examples and stick the answer to the job role

Consider asking yourself these inquiries to prepare your answer:

- What did a good day at work seem like in your previous role and why?
- How did you decide on your profession or field?

Example answer: “Making a real difference to save the lives of my patients and their families motivates me to keep working forward to save lives. I expect forward to seeing my patient’s reactions once we get a positive outcome that will change their lives forever. That’s why I became a nurse and why I’m pursuing a footing in pediatrics.”

6. What are your biggest strengths?

To answer this question, always share your most relevant technical and soft skills. While it should feel uncomfortable to speak highly of yourself, remember that this is often your opportunity to inform your interviewers what causes you to be a good candidate—and they need to listen to it.

To answer, follow the formula below:

Share one to some positive qualities and personal attributes: “I’ve always been a natural leader...”

Back them up with examples: “...I’ve exceeded my XYZs quarterly and are promoted twice within the past three years. I reminisce about those successes and know that I wouldn’t have reached them if I hadn’t built and led teams composed of highly skilled and diverse individuals. I’m happy with my ability to urge cross-functional groups on the identical page...”

Relate them to the role that you're interviewing: "...I've also regularly reviewed my management skills through reviews and candidate sessions with my team, and that I know continuing to make my leadership skills are a few things I need from my next role."

Top 10 Telephone Interview Questions you need to know

7. Do you have any questions?

This may be one of the foremost important questions asked during the interview process because it allows you to explore any topics that haven't been addressed and shows the interviewer you're serious about the role. Remember that you simply are interviewing the corporate too. Take time to ask the interviewer questions about their own experiences with the corporate, gain tips about how you'll be able to succeed if hired, and address any lingering questions you have got. Some examples include:

- What makes one love most about working for this company?
- According to you What is success?
- Tell me what challenges people typically face during this position?"
- How important is it that you just hire someone with XYZ qualities?
- Do you have any hesitations about hiring me?



8. Why do you want to work with this company?

Beware of normal answers, If your answer makes you sound like every other candidate then you're missing a chance to stand out from others. Do some research and point to something that makes the company unique that makes you to; talk about how you have been watching the company's growth. Point out the opportunities of your future growth in that company and explain how you can contribute company's growth; or share what's gotten you excited from your interactions with employees so far. Whatever you say, make sure to be specific and unique. If you can't find out why you would want to work at the organization you are interviewing with by the time you are well into the hiring process? It might be a bad impression telling you that this position is not the right fit.

9. How do you handle stress?

How you handle stressful situations is an indicator of your ability to resolve problems. Employers want to hire candidates who react to worry

constructively, so your answer to the present question must demonstrate personal growth. One of the common interview questions asked especially amidst the pandemic as everyone is juggling multiple things.

Spend a while brooding about how you approach to fret and give an example that communicates your ability to persevere in stressful situations.

Example answer: “I’m ready to stay calm once I specialize in the larger picture and break down my projects into smaller tasks. I always start by asking myself, “What is that the final word goal I’m trying to achieve?” From there, I make a listing of immediate and long-term action items with achievable but ambitious deadlines. Whether or not the large project is due tomorrow, I ask myself, ‘What’s something I can tackle within the next 30 minutes?’ Before I do know it, I’ve made significant progress which impossible project doesn’t seem so impossible.”

10. What Are You Passionate About?

You’re not a robot programmed to try and do your work so power down. You’re an individual, and if someone asks you this question in an interview, it’s probably because they need to induce you to understand you better. The solution can align directly with the sort of work you’d be doing in their role—like if, for instance, you’re applying to be a graphic designer and spend all of your free time creating simulations and data visualizations on social media.

But don’t be afraid to speak about a couple of hobbies that are different from your day-to-day work. Additional points if you’ll be able to “take it one step forward and connect how your passion would cause you to be an excellent candidate for the role you’re applying for,” Like if you’re a software developer who likes to develop code, you would possibly discuss how the flexibleness to be both creative and precise informs your approach to code.

11. Are You Willing to Relocate?

While this might sound sort of a simple yes-or-no question, it's often a little bit more complicated than that. The only scenario is one where you're totally receptive to moving and would be willing to try to do so for this chance. But if the answer is no or a minimum of not straight away, you'll reiterate your enthusiasm for the role, briefly explain why you can't move at this point, and offer an alternative, like working remotely or out of an area office. Sometimes it's not as clear-cut, and that's OK. You'll be able to remain put for XYZ reasons but would be willing to contemplate relocating for the proper opportunity.

12. How did you get to know about this position?

Another general interview question asked by many interviewers. Although it is a simple question, use this opportunity to pitch to stand out and show interest in the job role and company.

For example, you have found about the job role from your friend or professional contact, do mention their name and position and then share why you are interested in the job role and why you want to join the company. If you applied through a job portal or careers website, do mention that.

13. Why do you want this job role?

This is one of the most common interview questions asked in an interview. Interviewers here would generally like to hear about your passion for your work. Identify a few things that you like about the job role and highlight them, and then share why you love the company.

Example: I like this job role because I get a chance to work with many clients, which allows me to work on customer service and build networks to get more clients/leads.

14. What can you bring to this organization?

When interviewers ask this in an interview, they do not want to know just about your skills and achievements in your previous job roles. They want to know more about how you will face the challenges and adjust to the company's new culture than your background. Do thorough research about the company and the job role before you attend the interview. Connect your skill and experiences to what the company is looking for while answering this question.

15. Tell me about your most outstanding professional achievement

A lot of people feel shy or hesitate while answering this question; please do not do that. It is what you have achieved through hard work and dedication. One of the best ways to answer this question is using the STAR method - Situation, Task, Action, Results. Set up the situation and explain what you had to do, and give the interviewer the context. Then talk about how you implemented it and about the results.

Example: If you are working as a content marketer, your task was to write and rank blogs on the first page of SERP. Tell the interviewer about this and the process you followed while writing and optimizing for SEO and your results at last.

16. Tell me about a conflict you've faced at the workplace and how you dealt with it

This is a challenging question to answer for many candidates. As the first step, never say you have never had one. Be honest about the conflict you have had, and do not get into too many details like you would with a friend or a partner. Stay calm, keep it professional, and be honest (do not negatively criticize others involved in the conflict). Most interviewers ask this to check your honesty and how you would handle situations. Showcase that you learn from your mistakes and experiences.

17. Tell me about a situation/scenario where you demonstrated leadership skills

This is a tricky question. You do not need to be in a managerial/leadership position or act as a leader to showcase your leadership skills. This can be about an initiative taken, such as solving a problem, suggesting a new idea, and taking the lead in implementation or heading a project. You can again use the STAR method to highlight it effectively. Ensure you say the story clearly and leave no dots unconnected.

For more examples and tips on leadership questions, explore [Leadership Interview Questions and Answers](#).

18. Tell me about a time you disagreed with a decision at work/workplace

We all have disagreements at work, and it is ok to speak about them openly and honestly; do not be negative about them. An ideal answer would be to say that you have learned disagreements are bound to happen in the professional world, and it is absolutely fine. Then, tell about a scenario where you had one and close it on a high note with your learnings from it. The interviewer here is trying to check your adaptability and collaborative skills.

19. Tell me about your failure

The best way to answer this question is to be honest about your failure or a failed project/experience. It might not be correct if the organization does not give you a chance to fail or has no room for mistakes. You fail only to learn more. Say about an unintentional failure (pretty sure all failures are unintentional), but that was a risky situation and end it on a note telling about your learnings. The key here is how you explain it rather than what mistake you had made. So do not worry about past blunders while explaining.

20. Why are you leaving your current job?

This is one of the common interview questions but is a toughie. You can be sure that almost everyone will ask this. You need to keep things positive here and not take in a negative direction no matter what; you get nothing out of it. Focus more on how eager you are to learn new things from new job roles and why it is the best fit for you.

Example: I have worked majorly in organic marketing for over two years and would like to get into paid marketing and learn more about the same. This job role would be the best fit for me as it helps me learn both organic and paid marketing.

And if you were let go due to covid or restructuring or any other reason, be honest and say you were unfortunate that it happened. Do not stress on it too much unless the interviewer specifically asks you about it.

21. Why do you have a gap in your career?

Career gaps can happen due to various reasons ranging from being fired to maternal/paternal leaves. Amidst the pandemic, many have taken breaks from work due to mental health as well. It is not easy to explain as not every interviewer would have been through what you have been through and vice-versa. So you must make the interviewer feel your story. If the career gap is due to you getting fired or leaving for a professional reason, you can answer similar to the previous question.

If you had to take a break due to pregnancy, mental health, or any other personal issues, be open about it and end it on a positive note. Talk about how these made you learn more but do not get into too much detail if it makes you feel uncomfortable.

22. Tell me about something other than your resume

This is a question asked by hiring managers or interviewers when they want to know you more than your professional career. It is a good sign if they ask this question as they might have had a positive impression from your Resume/CV. Please answer this question in a positive way and tell a story that showcases your personality traits and can add your hobbies to keep it on a lighter note.

23. What are your salary expectations for this job role?

This is one of the most common interview questions asked in an interview, so be prepared and have a clear idea before the interview. Do thorough research on the salaries of job roles that you have applied for. You can use sites like Payschool, Glassdoor to search about the same - basis experience, skills, location, etc.

Coming to quoting salary, keep the following things in mind:

- First, request for their budgets or the salary range they have planned for the job role
- Always quote a salary range and not a fixed number (like 6 - 8 LPA)
- Take the negotiation at the end. Always try to discuss the salary negotiations at last, once you have known all about the job role.

24. How do you think we can improve as a company?

This is one of the most important interview questions, which might make or break many candidates. To answer this question, you need to first know about the organization/company in detail. What do they do? What are their services? Who is the founder? Latest News and a bit about the leadership team. Understanding the business externally will help you answer this question. Give the company good feedback and suggest improvements with some innovative ideas. Constructive feedback takes you a long way ahead of other candidates. Also, please explain why you told these things and ask some questions about the processes they follow to showcase your enthusiasm.

25. Any questions for us?

Another critical question where many candidates say, "Nothing much." It would be best if you always asked questions. The best way is to ask about their services or the latest projects they have implemented (you will find it on websites or social media for many companies) and ask about them in detail. This will show your enthusiasm towards the job role and your interest in the organization.

How many of these common interview questions you were prepared for? Do let us know in the comments.



Prepare

Prepare

Courses

Courses

Projects

Projects

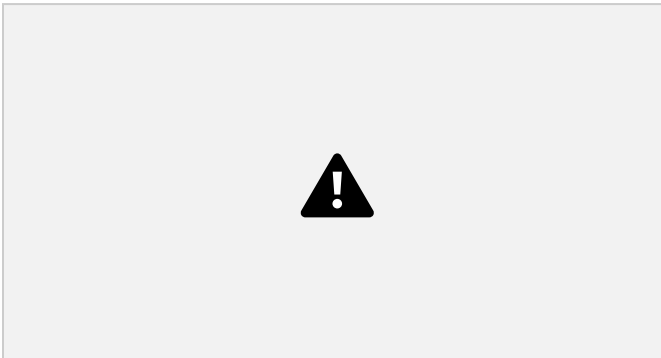
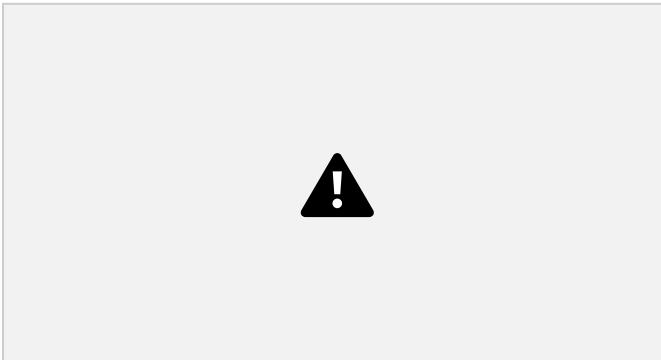
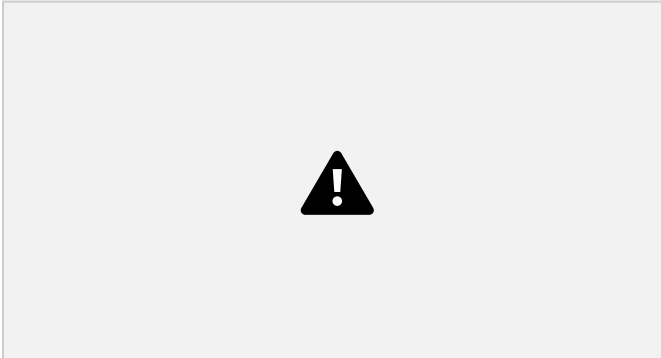
☐

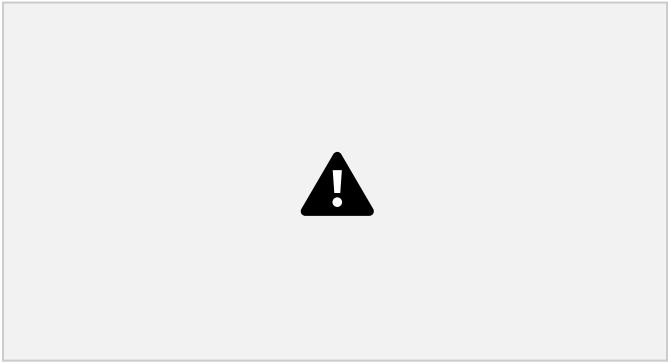
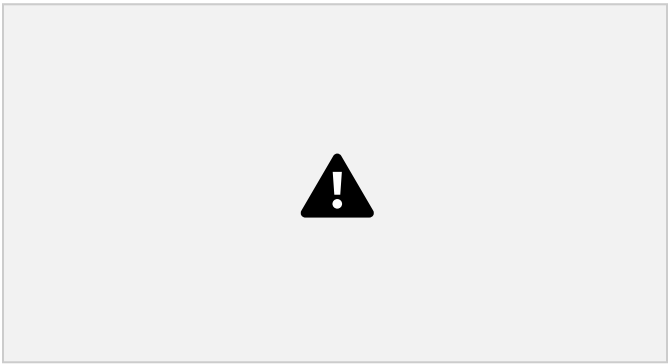
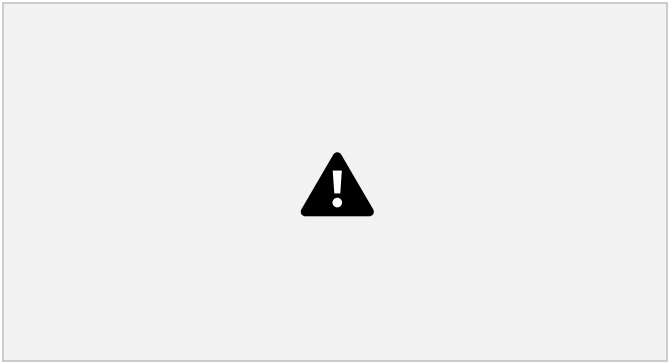
☐

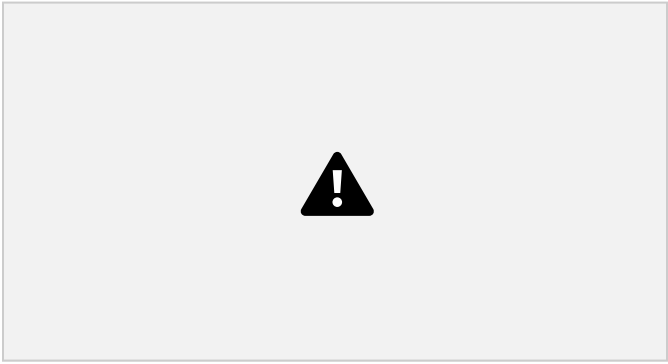
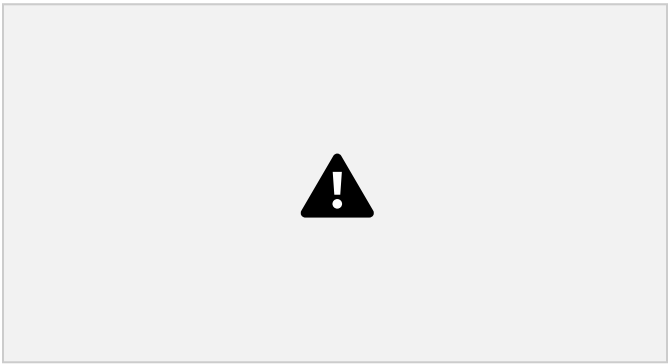
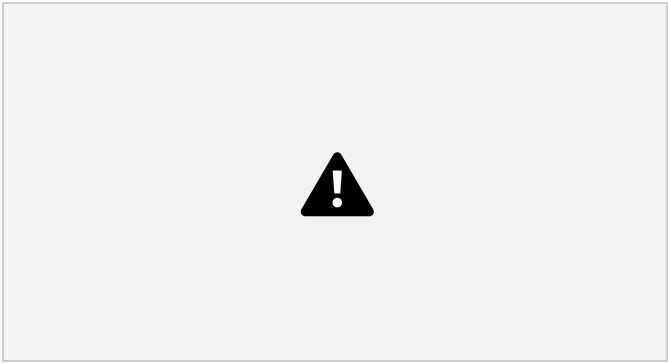
☐

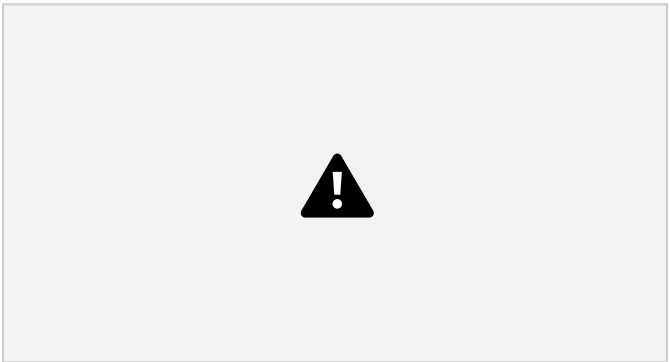
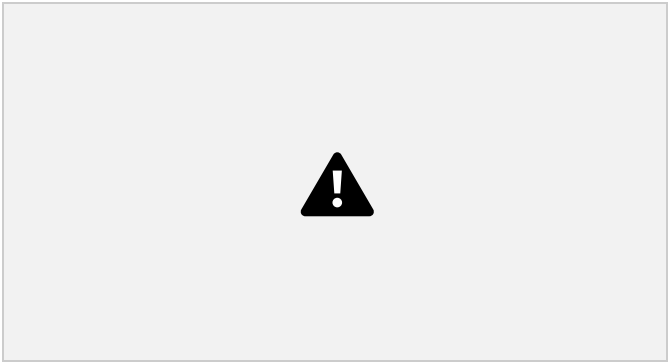
☐





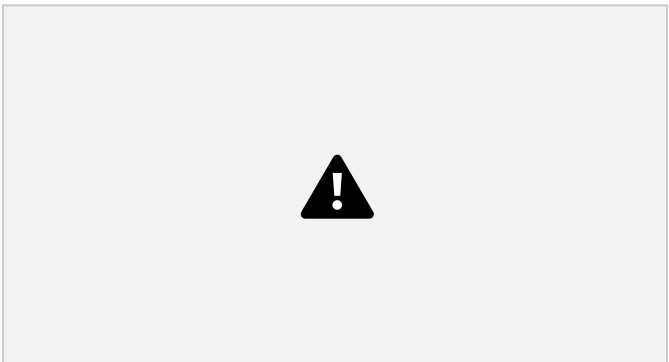


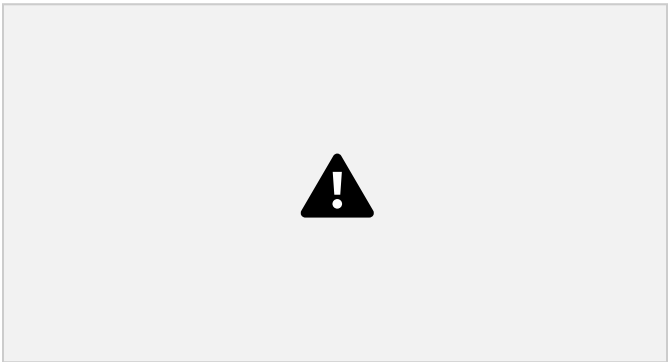
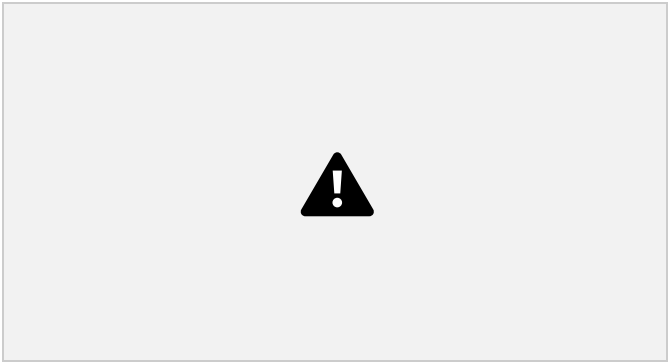




Skill Courses

[Skill Courses](#)





OffCampus

OffCampus



Login

Get Prime

Infosys SP and DSE Menu

Placement papers
Coding Questions
Syllabus
Recruitment Process
What is SP ?
SP Coding Questions
What is DSE ?
DSE Coding Questions
Get Hiring Updates
Contact Us

PREPINSTA PRIME

Prime Video



Apply For Jobs

Get Hiring Updates right in your inbox from Preplnsta



Get Hiring Updates

Infosys Coding Questions and Answer for SP and DSE role

Infosys SP and DSE Coding Questions with Solutions

Infosys SP and DSE Coding Questions and Answers are explained on this page. Coding round is one of the most important round for getting role for SP and DSE profile.

This section help you in analysing the difficulty level of the problem asked in Infosys. Practising Infosys coding question and answer help you in understanding the difficulty level and question pattern for coding round.



Important Note:

Below you will find Similar pattern based Infosys Coding Round Questions and their solutions in different languages. You must need to prepare the coding questions of Infosys exam for scoring good score.

Infosys SP and DSE Coding Questions Format:

- Coding round contains 3 questions that will have to be attended in 3 hours.
- Each questions have different difficulty level.
- There is one Easy problem based on Algorithm and Data structures.
- One of the problem is of Medium level and that problem is based on Greedy Algorithm.
- One is of Hard difficulty level, and usually based on Dynamic Programming.

No. Of Questions	Marks per question
Question 1	20
Question 2	30
Question 3	50

Infosys SP and DSE Coding Questions Analysis:

Question 1 –Easy level:

- Simple Question that can be solved by basic applications of Data Structures and Algorithms.

Question 2 – Medium Level:

Usually a question based on Greedy Algorithm

- Greedy algorithm is any algorithm that follows the problem solving heuristic of making the locally optimal choice at each stage.
- Types of greedy problems are:
 - Pure Greedy Algorithms
 - Orthogonal Greedy Algorithms
 - Relaxed Greedy Algorithms

Question 3 – Hard level:

Usually a question based on Dynamic Programming

- DP is an algorithmic technique for solving an optimisation problem by breaking it down into simpler subproblems and utilising the fact that the optimal solution to the overall problem depends upon the optimal solution to its subproblems.
- Principles of Dynamic Programming:
 - Breaking the problem down into subproblems and calculating their values. Next time, upon encountering the same subproblem, the value can be reused instead of recalculation
 - Avoid repeated work by remembering partial results, a common approach for performance enhancements.

[Also Checkout](#)

[Infosys SP and DSE Syllabus](#)

[Infosys SP and DSE Recruitment Process](#)

[Prime Course Trailer](#)

[Related Banners](#)

Get PrepInsta Prime & get Access to all 200+ courses offered by PrepInsta in One Subscription

[Get Prime](#)

Sample Infosys SP and DSE Coding Questions with Solutions

Question 1

Problem Statement: While playing an RPG game, you were assigned to complete one of the hardest quests in this game. There are n monsters you'll need to defeat in this quest.

- Each monster i is described with two integer numbers – $power_i$ and $bonus_i$. To defeat this monster, you'll need at least $power_i$ experience points. If you try fighting this monster without having enough experience points, you lose immediately. You will also gain $bonus_i$ experience points if you defeat this monster. You can defeat monsters in any order.
- The quest turned out to be very hard – you try to defeat the monsters but keep losing repeatedly. Your friend told you that this quest is impossible to complete.

Knowing that, you're interested, what is the maximum possible number of monsters you can defeat?

(Question difficulty level: Hardest)

Input:

- The first line contains an integer, n , denoting the number of monsters. The next line contains an integer, e , denoting your initial experience.
- Each line i of the n subsequent lines (where $0 \leq i < n$) contains an integer, $power_i$, which represents power of the corresponding monster.
- Each line i of the n subsequent lines (where $0 \leq i < n$) contains an integer, $bonus_i$, which represents bonus for defeating the corresponding monster.

In pu t	Out put	Output Description
2 12 3 78 13 0 10 0	2	- Initial experience level is 123 points. - Defeat the first monster having power of 78 and bonus of 10. Experience level is now $123+10=133$. - Defeat the second monster.
3 10 0 10 1 10 0 30 4 10 0 1 52 4	2	- Initial experience level is 100 points. - Defeat the second monster having power of 100 and bonus of 1. Experience level is now $100+1=101$. - Defeat the first monster having power of 101 and bonus of 100. Experience level is now $101+100=201$. - The third monster can't be defeated.

Java

Python

C++

Run

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int exp = s.nextInt();

        int[] monst = new int[n];
        int[] bonus = new int[n];

        for (int i = 0; i < n; i++) monst[i] = s.nextInt();
        for (int i = 0; i < n; i++) bonus[i] = s.nextInt();

        class Monster {
            int power, bonus;
            Monster(int power, int bonus) {
                this.power = power;
                this.bonus = bonus;
            }
        }

        Monster[] monsters = new Monster[n];
        for (int i = 0; i < n; i++) monsters[i] = new Monster(monst[i], bonus[i]);

        Arrays.sort(monsters, Comparator.comparingInt(m -> m.power));

        int count = 0;
        for (Monster m : monsters) {
            if (exp < m.power) break;
            exp += m.bonus;
            count++;
        }

        System.out.println(count);
    }
}
```

Question 2

Problem Statement:

Your birthday is coming soon and one of your friends, Alex, is thinking about a gift for you. He knows that you really like integer arrays with interesting properties.

He selected two numbers, N and K and decided to write down on paper all integer arrays of length K (in form $a[1], a[2], \dots, a[K]$), where every number $a[i]$ is in range from 1 to N , and, moreover, $a[i+1]$ is divisible by $a[i]$ (where $1 < i \leq K$), and give you this paper as a birthday present.

Alex is very patient, so he managed to do this. Now you're wondering, how many different arrays are written down on this paper?

Since the answer can be really large, print it modulo 10000.

Input:

- The first line contains an integer, n , denoting the maximum possible value in the arrays.
- The next line contains an integer, k , denoting the length of the arrays.

Input
t

Output
t

Output Description

2	2	The required length is 1, so there are only two possible arrays: [1] and
1		[2].

2	3	All possible arrays are [1, 1], [1, 2], [2, 2].
2		[2, 1] is invalid because 1 is not divisible by 2.

3	5	All possible arrays are [1, 1], [1, 2], [1, 3], [2, 2], [3, 3].
2		

Java

Python

C++

Run

```
import java.util.*;

class Main {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int k = sc.nextInt();
        System.out.println(count(n, k));
    }

    static int counter(int n, int k) {
        int count = 0;
        if (k == 1) return n;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (j % i == 0) count++;
            }
        }
        return count;
    }
}
```

```

static int count(int n, int k) {
    if (k == 1) return n;
    if (k == 2) return counter(n, k);
    int mid = k / 2;
    int x = count(n, k - mid);
    int y = counter(n, mid);
    return x + y - 1;
}
}

```



Question 3

Problem Statement:

You have an array A of N integers $A_1 A_2 \dots A_n$. Find the longest increasing subsequence $A_{i_1} A_{i_2} \dots A_{i_k}$

($1 \leq k \leq N$) that satisfies the following condition:

For every adjacent pair of numbers of the chosen subsequence $A_{i[x]}$ and $A_{i[x+1]}$ ($1 < x < k$), the expression $(A_{i[x]} \& A_{i[x+1]}) * 2 < (A_{i[x]} | A_{i[x+1]})$ is true

Note: '&' is the bitwise AND operation, '|' is the bit-wise OR operation

Input:

1. The first line contains an integer, N, denoting the number of elements in A.
2. Each line i of the N subsequent lines (where $0 \leq i < N$) contains an integer describing A_i .

Sample cases:

Input	Output	Output Description
5 15 6 5 12 1	2	One possible subsequence is: 5 12
6 9 17 2 15	2	One possible subsequence is: 2 15

5
2

7 3
17
16
12
2
8
17
17

One possible subsequence is: 2 8 17

C++

Java

Python

Run

```
class Main
{
    public static int LIS(int[] arr, int i, int n, int prev)
    {
        if (i == n) {
            return 0;
```



```

    }
    int excl = LIS(arr, i + 1, n, prev);

    int incl = 0;
    if (arr[i] > prev) {
        incl = 1 + LIS(arr, i + 1, n, arr[i]);
    }

    return Integer.max(incl, excl);
}

public static void main(String[] args)
{
    int[] arr = { 15, 6, 5, 12, 1 };

    System.out.print("The length of the LIS is "
        + LIS(arr, 0, arr.length, Integer.MIN_VALUE));
}
}

```

Sample Infosys SP and DSE Coding Questions

Question 4

Asked in Infosys Placment Paper – Feb 2022

Problem Statement :

You have been given a string S of length N. The given string is a binary string which consists of only 0's and '1's. Ugliness of a string is defined as the decimal number that this binary string represents.

Example:

- "101" represents 5.
- "0000" represents 0.
- "01010" represents 10.

There are two types of operations that can be performed on the given string.

- Swap any two characters by paying a cost of A coins.
- Flip any character by paying a cost of B coins
- flipping a character means converting a '1' to a '0' or converting a '0' to a '1'.

Initially, you have been given coins equal to the value defined in CASH. Your task is to minimize the ugliness of the string by performing the above mentioned operations on it. Since the answer can be very large, return the answer modulo 10^9+7 .

Note:

- You can perform an operation only if you have enough number of coins to perform it.
- After every operation the number of coins get deducted by the cost for that operation.

Input Format

- The first line contains an integer, N, denoting the number of character in the string

- The next line contains a string, S, denoting the the binary string
- The next line contains an integer, CASH, denoting the total number of coins present initially
- Next will contains an integer, A, denoting the cost to swap two characters.
- Then the next line contains an integer, B, denoting the cost to flip a character.

Constraints

- $1 \leq N \leq 10^5$
- $1 < \text{len}(S) \leq 10^5$
- $1 \leq \text{CASH} \leq 10^5$
- $1 \leq A \leq 10^5$
- $1 \leq B \leq 10^5$

Sample Input 1 :

```
4
1111
7
1
2
```

Sample Output 1 :

```
1
```

Explanation:

3 flips can be used to create "0001" which represents 1.

Sample Input 2:

6

111011

7

1

3

Sample Output 2:

7

Explanation:

First swap 0 with the most significant 1, then use flip twice first on index one and then on index two "111011"=>"0111111"=>"001111"=>"000111" the value represented is 7.

Sample Input 3:

6

111011

7

3

2

Sample Output 3:

3

Explanation:

Flip the 3 most significant characters to get "000011" : the value represented by this string is 3.N

C++

Java

Python

Run

```
import java.util.*;
class Main
{
    static String str;
    static int cash, n, a, b;
    static void swapf ()
    {
```

```

    char s[] = str.toCharArray ();
    int i = 0;
    for (int a = 0; a < s.length; a++)
        if (s[a] == '1')
        {
            i = a;
            break;
        }
    int j = s.length - 1;
    while (j > i)
    {
        if (cash < a)
            break;
        if (s[j] == '0')
        {
            if (s[i] == '0')
                i++;
            else
            {
                char temp = s[i];
                s[i] = s[j];
                s[j] = temp;
                cash -= a;
                j--;
            }
        }
        else
            j--;
    }
    str = new String (s);
}
static void flipf ()
{
    char s[] = str.toCharArray ();
    int i = 0;

    for (int a = 0; a < s.length; a++)
        if (s[a] == '1')
        {
            i = a;
            break;
        }
    while (cash >= b)
    {

```

```
        if (i == s.length)
            break;
        if (s[i] == '1')
        {
            s[i] = '0';
            i++;
            cash -= b;
        }
    }
    str = new String (s);
}
```

```
public static void main (String[]args)
{
    Scanner sc = new Scanner (System.in);
    n = sc.nextInt ();
    str = sc.next ();
    cash = sc.nextInt ();
    a = sc.nextInt ();
    b = sc.nextInt ();

    if (a < b)
    {
        swapf ();
        flipf ();
    }
    else
    {
        flipf ();
        swapf ();
    }
    System.out.println (Integer.parseInt (str, 2));
}
}
```



Question 5

Asked in Infosys Placment Paper – Feb 2022

Problem Statement :

Khaled has an array A of N elements. It is guaranteed that N is even. He wants to choose at most $N/2$ elements from array A. It is not necessary to choose consecutive elements. Khaled is interested in XOR of all the elements he chooses. Here, XOR denotes the bitwise XOR operation.

For example:

- If $A=[2,4,6,8]$, then khaled can choose the subset $[2,4,8]$ to achieve $XOR=(2 \text{ XOR } 4 \text{ XOR } 8)=14$.

Khaled wants to maximize the XOR of all the elements he chooses. Your task is to help khaled to find the max XOR of a subset that he can achieve by choosing at most $N/2$ elements?

Input format:

- The first line contains an integer, N , denoting the number of elements in A .
- Each line i of the N subsequent lines (where $0 \leq i \leq N$) contains an integer describing A_i .

Constraints

- $1 \leq N \leq 120$
- $1 \leq A[i] \leq 10^6$

Sample Input 1

2

1

2

Sample Output 1

2

Explanation:

$N=2$, $A=[1,2]$ khaled can choose the subset $[2]$. The xor of the elements in the subset is 2. And the number of elements in the subset is 1 which is less than $N/2$.

Sample Input 2

4

1

2

4

7

Sample Output 2

7

Explanation:

$N=4$, $A=[1,2,4,7]$ Khaled can choose the subset $[7]$. The xor of the elements in the subset is 7, and the number of elements in the subset is 1 which is less than $N/2$.

C++

Java

Python

Run

```
import java.util.*;
class Main
{
    public static void main (String[]args)
    {
        final int N = 120, M = 1 << 20;
        int dp[] = new int[M];
        char res[] = new char[M];

        Scanner sc = new Scanner (System.in);
        int n = sc.nextInt ();
        int arr[] = new int[n];

        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt ();

        for (int i = 1; i < M; i++)
            dp[i] = Integer.MAX_VALUE;

        for (int i = 0; i < n; ++i)
        {
            if (arr[i] == 0)
                continue;
            for (int j = 0; j < M; ++j)
                res[j] = 0;

            for (int k = 0; k < M; ++k) { if (res[k] == 1) continue; if (dp[k] > dp[k ^ arr[i]])
                dp[k] = dp[k ^ arr[i]] + 1;

            else if (dp[k ^ arr[i]] > dp[k])
                dp[k ^ arr[i]] = dp[k] + 1;

            res[k ^ arr[i]] = 1;
            }
        }

        int j = M - 1, k = n >> 1;
        while (dp[j] > k)
            --j;
        System.out.println (j);
    }
}
```



Sample Infosys SP and DSE Coding Questions and Answers

Question 6

Problem Statement :

Wael is well-known for how much he loves the bitwise XOR operation, while kaito is well known for how much he loves to sum numbers, so their friend Resli decided to make up a problem that would enjoy both of them. Resil wrote down an array A of length N, an integer K and he defined a new function called Xor- sum as follows

- $\text{Xor-sum}(x) = (x \text{ XOR } A[1]) + (x \text{ XOR } A[2]) + (x \text{ XOR } A[3]) + \dots + (x \text{ XOR } A[N])$

Can you find the integer x in the range $[0, K]$ with the maximum Xor-sum (x) value?

Print only the value.

Input format

- The first line contains integer N denoting the number of elements in A.
- The next line contains an integer, k, denoting the maximum value of x.
- Each line i of the N subsequent lines (where $0 \leq i \leq N$) contains an integer describing A_i .

Constraints

- $1 \leq N \leq 10^5$
- $0 \leq K \leq 10^9$
- $0 \leq A[i] \leq 10^9$

Sample Input 1

1
0
989898

Sample Output 1

989898

Explanation:

$$\text{Xor_sum}(0) = (0 \oplus 989898) = 989898$$

Sample Input 2

3

7

1

6

3

Sample Output 2

14

Explanation

$\text{Xor_sum}(4) = (4^1) + (4^6) + (4^3) = 14.$

Sample Input 3

4

9

7

4

0

3

Sample Output 3

46

Explanation:

$\text{Xor_sum}(8) = (8^7) + (8^4) + (8^0) + (8^3) = 46.$

C++

Java

Python

Run

```
import java.util.*;
class Main
{
    public static void main (String[] args)
    {

        Scanner sc = new Scanner (System.in);
        int n = sc.nextInt ();
        int k = sc.nextInt ();
        int arr[] = new int[n];

        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt ();

        int res = 0, max = Integer.MIN_VALUE;
```

```
for (int i = 0; i <= k; i++)  
{  
    res = 0;  
    for (int j = 0; j < n; j++)  
        res = res + (i ^ arr[j]);  
    max = Math.max (res, max);  
}  
System.out.println (max);  
}  
}
```



Sample Infosys SP and DSE Coding Questions

Question 7

Asked in Infosys Placement Paper – Sept 2021

Problem Statement :

One of the first lessons IT students learn is the representation of natural numbers in the binary number system (base 2) This system uses only two digits, 0 and 1. In everyday life we use for convenience the decimal system (base 10) which uses ten digits, from 0 to 9. In general, we could use any numbering system.

Computer scientists often use systems based on 8 or 16. The numbering system based on K uses K digits with a value from 0 to K-1. Suppose a natural number M is given, written in the decimal system To convert it to the corresponding writing in the system based on K, we successively divide M by K until we reach a quotient that is less than K

The representation of M in the system based on K is formed by the final quotient (as first digit) and is followed by the remainder of the previous divisions For example :

- If $M=122$ and $K=8$, 122 in base 10 = 172 in base 8 This means that the number
- In decimal system = 172 in octal system.
- 172 in base 8 = $1 \cdot 8^2 + 7 \cdot 8 + 2 = 122$

You made the following observation in applying the above rule of converting natural numbers to another numbering system

- In some cases in the new representation all the digits of the number are the same. For example 63 in base 10 = 333 in base 4

Given a number M in its decimal representation, your task is find the minimum base B such that in the representation of M at base B all digits are the same.

Input Format

- The first line contains an integer, M , denoting the number given

Constraints

- $1 \leq M \leq 10^{12}$

Sample Input 1 :

41

Sample Output 1 :

40

Explanation :

Here 41 in base 40. will be 11 so it has all digits the same, and there is no smaller base satisfying the requirements

Sample Input 2 :

34430

Sample Output 2 :

312

Explanation :

Here 34430 in base 312 will have all digits the same and there is no smaller base satisfying the requirements

C++

JAVA

Python

Run

```

import java.util.*;
class Main
{
    public static boolean convertBase (int m, int base)
    {

        int rem = m % base;
        m = m / base;

        while (m >= base && (m % base == rem))
            m = m / base;

        if (m == rem)
            return true;

        return false;
    }

    public static void main (String[] args)
    {
        Scanner sc = new Scanner (System.in);
        int m = sc.nextInt ();
        int base = 2;

        while (convertBase (m, base) != true)
            base++;

        System.out.println (base);
    }
}

```

Question 8

Asked in Infosys Placement Paper – Sept 2021

Problem Statement :

Andy wants to go on a vacation to de-stress himself. Therefore he decides to take a trip to an island. It is given that he has as many consecutive days as possible to rest, but he can only make one trip to the island. Suppose that the days are numbered from 1 to N . Andy has M obligations in his schedule, which he has already undertaken and which correspond to some specific days. This means that i th obligation is scheduled for day D_i . Andy is willing to cancel at most k of his obligations in order to take more holidays.

Your task is to find out the maximum days of vacation Andy can take by canceling at most K of his obligations.

Input Format

- The first line contains an integer N , denoting the total number of days
- The next line contains an integer M denoting the total number of obligations.
- The next line contains an integer K denoting the largest number of obligations he could cancel
- Each line i of the M subsequent lines (where $0 \leq i \leq M$) contains an integer describing D_i .

Constraints

- $1 \leq N \leq 10^6$
- $1 \leq M \leq 2 \cdot 10^6$
- $1 \leq K \leq 2 \cdot 10^6$

- $1 \leq D[i] \leq 10^6$

Sample Input 1:

10

5

2

6

9

3

2

7

Sample Output 1 :

5

Explanation:

Here he could cancel his 3rd and 4th obligation which makes vacation length 5.

Sample input 2:

7

2

0

3

4

Sample Output 2:

3

Explanation:

Here he could not cancel any obligation since $K=0$, so the vacation length is 3.

C++

Java

Python

Run

```
import java.util.*;
class Main
{
    public static void main (String[] args)
```

```

{

Scanner sc = new Scanner (System.in);
int n = sc.nextInt ();
int m = sc.nextInt ();
int k = sc.nextInt ();

int arr[] = new int[n];

for (int i = 0; i < m; i++) arr[i] = sc.nextInt (); int ans = 0; Arrays.sort (arr); if (k > 0)
{
    for (int i = k + 1; i <= m + 2; i++)
        ans = Math.max (ans, arr[i] - arr[i - k - 1] - 1);
}
else
{
    int j = 0;
    while (arr[j] == 0)
        j++;
    int count = 0;

    for (int i = 1; i <= n; i++)
    {
        count++;
        if (j < n && (i == arr[j]))
        {
            count = 0;
            j++;
        }
        ans = Math.max (count, ans);
    }
}
System.out.println (ans);
}
}

```




Infosys SP and DSE Coding Questions with Solutions

Question 9

Problem Statement :

You need to build a road in a rugged terrain. You know the sea level of each segment of the rugged terrain, i.e., the i -th segment is L_i meters from sea level.

You need to transform the terrain into a strictly downward sloping terrain for the road, i.e., for each i -th segment where $2 \leq i \leq N$, resultant $L_{i-1} > L_i$. To do so, you employ a powerful digging team to help you dig and reduce the sea level of the segments. On day D , the team can reduce the sea level for each segment that you scheduled that day by $2D-1$ meters each.

You are allowed to assign the team to dig on multiple segments and/or dig on the same segments for multiple days.

Your task is to find the minimum number of days needed to transform the terrain as per your requirements.

Input Format:

N :: INTEGER

The first line contains an integer, N, denoting the number of elements in L. $N :: 1 \rightarrow 10^5$

L :: INTEGER ARRAY

Each line i of the N subsequent lines (where $0 < i \leq N$) contains an integer describing L_i , the sea level of the i -th segment. $L[i] :: -10^9 \rightarrow 10^9$

Sample Input 1:

2

3

3

Sample Output 1 :

1

Sample input 2:

2

5

-3

Sample Output 2:

0

C++

Java

Python

Run

```
import java.util.*;
import java.lang.Math;
class Main{

    public static void main(String[] args)
    {

        Scanner s = new Scanner(System.in);
        int n = s.nextInt();
        int[] a = new int[n];
        for(int i =0; i< n; i++)
        {
            a[i] = s.nextInt();
        }
        int max_dig = 0;
        for(int i =0; i< n-1; i++)
        {
            if(a[i]<=a[i+1])
            {
                max_dig = Math.max(max_dig,(a[i+1]-a[i]+1));
                a[i+1] = a[i] -1;
            }
        }
        int ans = (int)Math.sqrt(max_dig);
        System.out.println(ans);
    }
}
```

Question 10

Problem Statement :

You are given an array of size N. You need to change this array into a mountain. By mountain we mean, the either ends of the array should have equal elements. Then as we move towards the

middle from both ends, the next element is just one more than the previous one. So, it would have a peak in the middle and decrease if you go towards either end, just like a mountain.

Examples of mountains are [1, 2, 3, 2, 1] or [6, 7, 8, 8, 7, 6]. But the array [1, 2, 4, 2, 1] is not a mountain because from 2 to 4 the difference is 2. The array [1, 2, 3, 1] is also not a mountain because the elements 2 and 3 are not equal from both ends.

You need to find the minimum number of elements that should be changed to make the array a mountain. You can make the elements negative or zero as well.

Input Format

N :: INTEGER

The first line contains an integer, N, denoting the number of elements in array. N :: 1 -> 10⁵

array :: INTEGER ARRAY

Each line i of the N subsequent lines (where $0 \leq i < N$) contains an integer describing i-th element of array. array[i] :: 1 -> 10⁶

Sample Input 1:

5
1
2
3
4
5

Sample Output 1 :

2

Sample input 2:

9

1

1

1

2

3

2

1

1

1

Sample Output 2:

4

C++

Java

Python

Run

```
import java.util.Scanner;
class Main{
public static int mountain(int[] arr, int n){
    int count=0;
    if(n%2==0){
        int m2 = n/2;
        int m1 = m2-1;

        if(arr[m1] == arr[m2]){
            for(int i =m1-1; i>=0; i--){
                if(arr[i] != (arr[i+1] -1)){
                    count++;
                    arr[i] = (arr[i+1]-1);
                }
            }
        }
    }
}
```

```

    }
}
}
    else if(arr[m1] != arr[m2]){
        arr[m2] = arr[m1];
        for(int i =m1-1; i>=0; i--){
            if(arr[i] != (arr[i+1] -1)){
                count++;
                arr[i] = (arr[i+1]-1);
            }
        }
    }

}

    int j = m1-1;
    for(int i =m2+1; i<n; i++){ if(arr[i] != arr[j]){ count ++; } j--; } return count; } else{ int mid = n/2;
for(int i = mid-1; i>=0; i--){
    if(arr[i] != (arr[i+1]-1)){
        count++;
        arr[i] = (arr[i+1]-1);
    }
}
}
    int j = mid-1;
    for(int i = mid+1; i < n; i++){
        if(arr[i] !=arr[j]){
            count++;
        }
        j--;
    }
    return count;
}

}

public static void main(String[] args) {
Scanner s = new Scanner(System.in);
    int n = s.nextInt();
    int arr[] = new int[n];
    for(int i =0; i < n; i++){
        arr[i]= s.nextInt();
    }
    System.out.println(mountain(arr,n));
}
}

```

Question 11

Problem Statement :

You have an interesting string S of length N . It is interesting because you can rearrange the characters of this string in any order. You want to cut this string into some contiguous pieces such that after cutting, all the pieces are equal to one another.

You can't rearrange the characters in the cut pieces or join the pieces together. You want to make the number of pieces as large as possible. What is the maximum number of pieces you can get?

Note: You can observe that you may not want to cut the string at all, therefore the number of pieces is 1. Hence, the answer always exists.

Input Format

$S :: \text{STRING}$

The first line contains a string, S , denoting the string.

$\text{length}(S) :: 1 \rightarrow 2 * 10^5$

Sample Input 1:

zzzzz

Sample Output 1 :

5

Sample input 2:

ababcc

Sample Output 2:

2

Sample input 2:

abccdcabacda

Sample Output 2:

2

C++

Java

Python

Run

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        String s;
        Scanner sc = new Scanner(System.in);
        s = sc.next();
        System.out.print(ans(s));
    }
    static int ans(String s) {
        HashMap<Character, Integer> map = new HashMap<>();
        for (int i = 0; i < s.length(); i++) {
            if (!map.containsKey(s.charAt(i))) {
                map.put(s.charAt(i), 1);
            } else {
                map.put(s.charAt(i), map.getOrDefault(s.charAt(i), 1)+1);
            }
        }
        int key = 100000;
        int val = 0;
        for (int i = 0; i < s.length(); i++) { if (key > map.get(s.charAt(i))) {
            val = map.get(s.charAt(i));
        }
    }
}
```



```

        key = map.get(s.charAt(i));
        // we will get the smallest number.
    }
}
return val;
}
}

```

Question 12

Problem Statement :

Today you decided to go to the gym. You currently have energy equal to E units. There are N exercises in the gym. Each of these exercises drains A_i amount of energy from your body.

You feel tired if your energy reaches 0 or below. Calculate the minimum number of exercises you have to perform such that you become tired. Every unique exercise can only be performed at most 2 times as others also have to use the machines.

If performing all the exercises does not make you feel tired, return -1.

Input Format

$E :: \text{INTEGER}$

The first line contains an integer, E , denoting the Energy.

$E :: 1 \rightarrow 10^5$

$N :: \text{INTEGER}$

The next line contains an integer, N , denoting the number of exercises. $N :: 1 \rightarrow 10^5$

A :: INTEGER ARRAY

Each line i of the N subsequent lines (where $0 \leq i < N$) contains an integer describing the amount of energy drained by i -th exercise.

A[i] :: 1 -> 10^5

Sample Input 1:

6

2

1

2

Sample Output 1 :

4

Sample input 2:

10

2

1

2

Sample Output 2:

-1

Sample input 3:

2

3

1

5

2

Sample Output 3:

1

C++

Java

Python

Run

```
import java.util.Scanner;
public class Main {
    public static int solution(int nExercise,int[] arr,int energy){
        int sum = 0;
        for(int i = 0; i< nExercise; i++) {
            if(arr[i]==energy)
                return 1;
        }
        for(int i =0; i<nExercise; i++){ sum = sum + arr[i]; if(arr[i]== energy){ return 1; } if(sum >= energy){
            return i+1;
        }
        if(2*sum >= energy){
            return 2*(i+1);
        }
    }
    //if sum is not equal to the energy.
    return -1;
}
```

```
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int energy = s.nextInt();
        int nExercise = s.nextInt();
        int[] arr= new int[nExercise];
        for (int i = 0; i < nExercise; i++)
        {
            arr[i] = s.nextInt();
        }
    }
```

```
    int ans = solution(nExercise,arr,energy);
    System.out.println(ans);
}
```



Infosys SP and DSE Coding Questions with Solutions

Question 13

Problem Statement :

There is a battle between heroes and villains going on. You have M heroes, all of them have the same health H . There are N villains, health of the i -th villain is V_i .

When a hero, with health H battles a villain with health V_i , one of the three scenarios can happen:

if $H > V_i$: The villain is defeated, and the health of the hero is decreased by V_i if $H < V_i$: The villain wins, his health is not affected, and the hero is no longer able to fight. if $H = V_i$: Both are considered defeated, and neither can fight.

The heroes start fighting villains one by one in the same order, first villain 1 then villain 2 and so on. It might be possible that before defeating all the villains, all the heroes are defeated. Therefore, to ensure the victory of the heroes, you want to remove some villains from the front.

Your task is to find the minimum number of villains you need to remove from the front such that the victory of the heroes is guaranteed.

Note: If in the last battle, both the hero and villain are defeated and no more heroes or villains remain, it would still be considered a victory since all the villains are defeated.

Input Format

N :: INTEGER

The first line contains an integer, N, denoting the number of villains N :: 1 -> $2 \cdot 10^5$

M :: INTEGER

The next line contains an integer, M, denoting the number of heroes M :: 1 -> $2 \cdot 10^5$

H :: INTEGER

The next line contains an integer, H, denoting the health of each of the heroes H :: 1 -> 10^9

array :: INTEGER ARRAY

Each line i of the N subsequent lines (where $0 \leq i < N$) contains an integer describing the health of each of the villains.

array[i] :: 1 -> 10^9

Sample Input 1:

4

4

3

3

1

3

3

Sample Output 1 :

0

Sample input 2:

5

3

3

1

2

3

1

1

Sample Output 2:

0

Sample input 3:

5

1

4

1

2

3

1

3

Sample Output 3:

3

C++

Java

Python

Run

```
import java.util.Scanner;
class Main{
public static int ans(int[] arr, int n, int m, int h){
    int total = 0;
    for(int i =0; i<n; i++){
        total = total +arr[i];
    }
    if(total <= m*h){ return 0; } else{ int sumh = 0; int totalhero = m*h; for(int i=n-1; i>0; i--){
        sumh= sumh+arr[i];
        if(sumh >= totalhero){
            return i;
        }
    }
}

}
return 0;

}

public static void main(String[] args) {
    Scanner s = new Scanner(System.in);
    int n = s.nextInt();
    int m = s.nextInt();
    int h = s.nextInt();
    int arr[] = new int[n];
    for(int i =0; i<n; i++){
        arr[i] = s.nextInt();//health of villians
    }
    System.out.println(ans(arr,n,m,h));

}
}
```

Get over 200+ course One

Subscription

Courses like AI/ML, Cloud Computing, Ethical Hacking, C, C++, Java, Python, DSA (All Languages), Competitive Coding (All Languages), TCS, Infosys, Wipro, Amazon, DBMS, SQL and others

Get Prime

Course List

Checkout list of all the video courses in Preplnsta Prime
Subscription

Checkout

Login/Signup to comment



Maths

#include

using namespace std;

int main()

{

int n, k, m;

cin >> n >> k;

vector arr (n);

for (int i = 0; i < n; i++)

arr[i] = m;

}

int my_sum=accumulate(arr.begin(), arr.end(), 0);

for(int i=0; i<my_sum; i++)

my_sum+=arr[i];

}

}

}

cout<<my_sum;

}

Log in to Reply



Rohit

When will be open hiring for infosys SP and DSE roles for the 2023 batch

Log in to Reply



anup

for last question of array vacation ,for input 10 5 2 2 4 5 7 9 it should have given 4 instead of 5 as o/p

Log in to Reply



anushapavuluri66

Thanks, sir I got placed in Infosys as a Specialist Programmer due to your Prime Video Course thanks

Log in to Reply



HelpPrepInsta

Hey Guys,

If you are looking for Infosys SP and DSE preparation course, please check out this link :

[Click here](#)

Log in to Reply



Viswatej

Hey what will be the first test include which will be for 180 minutes could you brief me please

Log in to Reply



pvdv

Sir your platform has been very useful to me in learning and getting placed!

Log in to Reply



HelpPrepInsta

Thank you so much Likhitha.

We wish you all the best!

Also, guys if you are looking for preparation material which Covers whole Python, Java DSA, Competitive Coding etc and past year's Infosys InfyTQ coding questions, [click here](#)

Log in to Reply



Muskan

From where can I get off campus Drive updates?

Log in to Reply



HelpPrepInsta

Hey Muskan, You can get all the off campus updates from here.

[Click here](#)

[Log in to Reply](#)



savitri

Good platform form to know about coding

[Log in to Reply](#)



PrepInsta.com

No.1 and most visited website for Placements in India.

We help students to prepare for placements with the best study material, online classes, Sectional Statistics for better focus and Success stories & tips by Toppers on PrepInsta.

PrepInsta Powered by Metis Eduventures Private Limited

© 2026 Prep Insta

Support

[PrepInsta Prime](#)

[PrepInsta Prime Max](#)

[Contact Us](#)

[About Us](#)

[Refund Policy](#)

[Privacy Policy](#)

[Services](#)

[Disclaimer](#)

[Terms and Conditions](#)

Companies

[Accenture](#)

[Cognizant](#)

[MindTree](#)

[VMware](#)

[CapGemini](#)

[Deloitte](#)

[Wipro](#)

[Microsoft](#)

[TCS](#)

[Infosys](#)

[Oracle](#)

[HCL](#)

[TCS Ninja](#)

[IBM](#)

All Exams Dashboards

[CoCubes Dashboard](#)

[eLitmus Dashboard](#)

[HirePro Dashboard](#)

[MeritTrac Dashboard](#)

[Mettl Dashboard](#)

[DevSquare Dashboard](#)

Get In Touch

[Instagram](#)

[Linkedin](#)

[Youtube](#)

[Telegram](#)

[Facebook](#)

[Twitter](#)

Get In Touch

support@prepinsta.com

[+91-8448440710](tel:+91-8448440710)

[Text us on Whatsapp/Instagram](#)

Partner Companies

[Career247](#)

[Privacy Policy](#) | [Copyright © 2026 Prep Insta](#)



30+ Companies are Hiring

Get Hiring Updates right in your inbox from Preplnsta

Select your Passing out Year

2022

2024

2026

2023

2025

2027

2028

2029

Others

Get Hiring Updates


IBM Powered Courses

Data Analytics
Powered by IBM



Business Analytics
Powered by IBM



Data Science
Powered by IBM



Checkout Now